



中国研究生创新实践系列大赛  
“华为杯”第二十届中国研究生  
数学建模竞赛

学 校

北京理工大学

---

参赛队号

23100070010

---

队员姓名

1.

吴志伟

---

2.

彭皓崧

---

3.

郭睿琦

---

中国研究生创新实践系列大赛  
“华为杯”第二十届中国研究生  
数学建模竞赛

题 目： DFT 类矩阵的整数分解逼近：解析与优化方法

---

摘 要：

本文研究了 DFT 类矩阵的整数分解逼近问题。基于问题对逼近矩阵的稀疏性约束和元素约束，我们分别提出了 DFT 矩阵的高精度解析逼近方法和基于稀疏矩阵逼近的整数规划模型优化方法。稀疏矩阵逼近问题属于 NP-hard 问题，难以在多项式时间内求解。我们针对 5 个问题分别给出了定制的求解方法，经过适当改进和约束放松后，阐述了他们对任意 DFT 类矩阵的精确逼近可行性，以建立对 5 个问题的两套统一解决方案，能够兼顾精确性和计算复杂度。

针对问题一，我们采用[Cooley-Tukey 基-2 分解](#)方法，利用其对  $2^t$  规模 DFT 矩阵的迭代分解性质，将 DFT 矩阵按蝶形操作完美分解为  $\log_2(N)$  个稀疏矩阵和一个排列矩阵的乘积，实现对任意  $N$  阶 DFT 矩阵的估计误差为 0。构造的蝶形因子在满足约束 1 的同时成为解析逼近的良好切入点。在不需要满足约束 2 的条件下，分解  $\mathbf{F}_4$ ,  $\mathbf{F}_8$  得到的稀疏矩阵具有 0 硬件复杂度。

针对问题二，我们提出了[整数映射约束的层次分解](#)算法来解决无稀疏性约束下的限定实部和虚部取值范围的矩阵逼近问题。该算法给定一个需要分解的矩阵和矩阵因子的个数，通过基于梯度的[邻近交替线性化最小化](#)的方法迭代更新各个矩阵因子，在每一步更新后利用投影算子将矩阵因子的元素映射到限定的范围内。为了提升矩阵分解的近似精度，采用分层和递归的思想设计层次分解算法，将目标矩阵多次两两分解提高逼近程度。该方法能对  $\mathbf{F}_4 \sim \mathbf{F}_8$  达到 0.781 ~ 0.969 的 RMSE，并且控制分解矩阵的个数为  $\log_2 N$  和 0 硬件复杂度。

针对问题三，面对约束条件较多的情况，我们分别从解析逼近和优化逼近的方向提出基于[遍历性定理](#)和基于二叉树节点的[蝶形稀疏矩阵分层量化](#)分解算法。对于解析逼近，通过继承来自 C-T 基-2 分解得到的先验稀疏矩阵，我们结合[Weyl 均匀分布定理](#)提出[遍历性搜索算法](#)。算法能够实现仅使用满足约束 2 的矩阵对先验稀疏矩阵实现高精度逼近 (RMSE 约为 0.06)，并保留了与硬件复杂度的权衡空间。对于优化逼近，通过定义

蝶形因子与蝶形结构矩阵, 提出了[固定支持矩阵量化分解](#)算法和[最优- \$\beta\$  贪心策略](#)以实现 DFT 矩阵的分层量化分解。算法对  $N = 2^t, t = 2, \dots, 5$  DFT 矩阵的 RMSE 分别为 0, 0.16, 0.33, 和 0.38, 硬件复杂度均为 0。

针对问题四, 我们证明任意  $\mathbf{F}_{N_1} \otimes \mathbf{F}_{N_2}$  都可以写成

$$\begin{aligned} & (\mathbf{A}_{t_1, t_1} \otimes \mathbf{I}_{N_2}) (\mathbf{A}_{t_1, t_1-1} \otimes \mathbf{I}_{N_2}) \dots (\mathbf{A}_{t_1, 1} \otimes \mathbf{I}_{N_2}) (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2, t_2}) \\ & \times (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2, t_2-1}) \dots (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2, 1}) (\mathbf{P}_{N_1}^\top \otimes \mathbf{P}_{N_2}^\top) \end{aligned} \quad (0.1)$$

的形式, 并且任意  $\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2, n}$  或者  $\mathbf{A}_{t_1, m} \otimes \mathbf{I}_{N_2}$  都满足约束 1。随后分别使用遍历性搜索方法和[丢番图逼近](#)方法得到 0.0682 和  $4.0605 \times 10^{-4}$  的 RMSE。

针对问题五, 我们将遍历性搜索和丢番图逼近方法相结合, 把解析法拓展到了对任意 DFT 矩阵的逼近, 并且给出了其能以任意精度逼近的数学证明, 对于  $\mathbf{F}_{16}$  逼近的 RMSE 达到 0.0109。对于优化方法, 我们提出了[分布式遗传算法的整数规划改进策略](#), 通过建立以元素指数为决策变量的整数规划模型, 应用遗传算法实现对 DFT 矩阵的整数分解。结果表明, 对比问题 4 的结果, 优化方法对  $\mathbf{F}_8$  和  $\mathbf{F}_{16}$  的 RMSE 提升至 0.13 和 0.17。值得说明的是, 两种算法都具有高度可扩展性, 以应用于大规模 DFT 矩阵的分解。

总体来说, 我们分别从解析逼近和优化逼近的两个方向给出了对所有问题的求解算法, 两种算法的优势明显, 可以根据实际应用需要进行选择。

**关键词：** DFT 库利 - 图基 FFT 蝶形因子 层次分解 丢番图逼近

## 目录

|          |                         |           |
|----------|-------------------------|-----------|
| <b>1</b> | <b>问题重述</b>             | <b>5</b>  |
| 1.1      | 问题背景                    | 5         |
| 1.2      | 问题提出                    | 6         |
| 1.3      | 模型总览                    | 6         |
| <b>2</b> | <b>模型的假设与定义</b>         | <b>8</b>  |
| <b>3</b> | <b>符号说明</b>             | <b>8</b>  |
| <b>4</b> | <b>问题 1 的分析与求解</b>      | <b>9</b>  |
| 4.1      | 问题 1 分析                 | 9         |
| 4.2      | 递归库利-图基快速傅里叶变换算法        | 11        |
| 4.2.1    | 蝶形矩阵                    | 11        |
| 4.2.2    | C-T 基-2 分解              | 12        |
| 4.3      | 矩阵逼近结果评估                | 13        |
| 4.3.1    | Frobenius 范数误差          | 13        |
| 4.3.2    | 硬件复杂度                   | 13        |
| <b>5</b> | <b>问题 2 分析与求解</b>       | <b>15</b> |
| 5.1      | 问题 2 分析                 | 15        |
| 5.2      | 整数映射约束的层次分解             | 15        |
| 5.2.1    | 柔性近似多层稀疏变换              | 15        |
| 5.2.2    | 层次分解                    | 19        |
| 5.3      | 问题 2 求解                 | 19        |
| <b>6</b> | <b>问题 3 分析与求解</b>       | <b>22</b> |
| 6.1      | 问题 3 分析                 | 22        |
| 6.2      | $\mathbb{F}_8$ 的结构分析与求解 | 22        |
| 6.2.1    | 遍历性分析                   | 23        |
| 6.2.2    | 遍历性定理逼近求解               | 26        |
| 6.3      | 蝶形稀疏矩阵分层量化分解            | 27        |
| 6.3.1    | 蝶形因子与蝶形结构               | 27        |



|             |                   |           |
|-------------|-------------------|-----------|
| 6.3.2       | 算法与求解             | 28        |
| 6.3.3       | 算法结果分析            | 30        |
| <b>7</b>    | <b>问题 4 分析与求解</b> | <b>35</b> |
| 7.1         | 问题 4 分析           | 35        |
| 7.2         | 丢番图逼近             | 35        |
| 7.3         | 丢番图逼近的整数规划问题求解    | 37        |
| 7.4         | 问题 4 的高精度求解       | 37        |
| <b>8</b>    | <b>问题 5 分析与求解</b> | <b>39</b> |
| 8.1         | 问题 5 分析           | 39        |
| 8.2         | 任意 DFT 矩阵的任意精度逼近  | 39        |
| 8.3         | 分布式遗传算法的整数规划改进策略  | 42        |
| <b>9</b>    | <b>模型评价</b>       | <b>43</b> |
| 9.1         | 模型优点              | 43        |
| 9.2         | 模型缺点              | 43        |
| <b>附录 A</b> | <b>问题二附录</b>      | <b>45</b> |
| <b>附录 B</b> | <b>问题四附录</b>      | <b>53</b> |
| <b>附录 C</b> | <b>主要实验代码</b>     | <b>54</b> |
| 3.1         | 问题 2 代码           | 54        |
| 3.2         | 问题 3 代码           | 61        |
| 3.3         | 问题 4 代码           | 67        |
| 3.4         | 问题 5 代码           | 68        |
| <b>参考文献</b> |                   | <b>73</b> |

# 1 问题重述

## 1.1 问题背景

在数字信号处理领域，离散傅里叶变换（DFT）是最基础且最关键的工具之一。他为我们提供了从时域到频域的转换手段，使得频域分析和操作成为可能。然而，传统的 DFT 计算方法在大数据集上显得效率低下，硬件复杂度高达  $O(N^2)$ 。为了解决这一问题，矩阵分解技术在快速傅里叶变换（FFT）中得到了应用，将硬件复杂度降低到了  $O(N \log N)$ ，该算法是一个里程碑式的发明，由于 FFT 的高效性，他使得许多实际应用成为可能。例如，实时音频处理、图像处理、无线通信、生物医学等领域都得益于 FFT 的高速计算。

在芯片设计中，DFT 的硬件需求与其算法复杂度和数据取值范围直接相关。更高的算法复杂度和更大的数据范围会增加硬件的复杂性。但随着无线通信技术的进步，如更大的天线阵列、更多的通道和更宽的通信带宽，FFT 的需求也随之增加，进而增加了在专用芯片上实现 FFT 的硬件资源需求。为了进一步优化硬件资源使用，一个策略是将 DFT 矩阵分解为矩阵连乘的形式，这种分解通常被称为“矩阵分解”，不仅可以降低硬件复杂度，还有助于并行处理和流水线设计，从而提高硬件实现的效率。

具体来说，给定一个长度为  $N$  的复数序列  $x_n$ ，其 DFT 为  $X_k$ ，定义为：

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-j\frac{2\pi}{N}kn} \quad (1.2)$$

写成矩阵形式为：

$$\mathbf{X} = \mathbf{F}_N \mathbf{x} \quad (1.3)$$

其中  $\mathbf{x} = [x_0, x_1 \dots x_{N-1}]^T$  为时域信号向量， $\mathbf{X} = [X_0, X_1 \dots X_{N-1}]^T$  为变换后的频域信号向量，因此，整个 DFT 变换矩阵  $F_N$  可以表示为：

$$\mathbf{F}_N = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & w & w^2 & \dots & w^{N-1} \\ 1 & w^2 & w^4 & \dots & w^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w^{N-1} & w^{2(N-1)} & \dots & w^{(N-1)(N-1)} \end{bmatrix} \quad (1.4)$$

DFT 类矩阵的整数分解逼近是一种尝试将 DFT 矩阵近似为整数矩阵的方法，在传统的 DFT 计算中，需要进行复数乘法，这涉及到实部和虚部的乘法和加法。通过整数分解逼近，可以将这些复数乘法转化为整数乘法或更简单的位移操作，从而减少硬件复杂度。例如在文献 [1] 中，作者提出了一种低复杂度的无乘法近似的 8 点 FFT，只需要 26 次加法运算。总的来说，DFT 类矩阵的整数分解逼近旨在通过简化 DFT 的计算过程来降低硬件复杂度，使其更适合于硬件实现。

## 1.2 问题提出

本题要求对给定偶数阶维度的离散傅里叶矩阵（DFT）使用矩阵连乘拟合的思路进行拟合，以代替 FFT 的计算方法降低硬件复杂度。给定已知的  $N$  维 DFT 矩阵，要求设计  $K$  个矩阵  $\mathcal{A} = \mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_K$ ，使得  $\beta \mathbf{F}_N$  和  $\prod_k^K \mathbf{A}_k$  在 Frobenius 范数意义下尽可能地接近，即

$$\min_{\mathcal{A}, \beta} \text{RMSE}(\mathcal{A}, \beta) = \frac{1}{N} \sqrt{\|\beta \mathbf{F}_N - \prod_k^K \mathbf{A}_k\|_F^2}$$

，其中  $\beta$  是实值矩阵缩放因子，并根据约束条件进行设计。此外，还考虑乘法器的硬件复杂度  $C = q \times L$ ， $q$  表明  $\mathbf{A}_k$  中元素的取值范围。 $L$  表示复数乘法次数。考虑以下两种约束条件：

**约束 1：**限定  $\mathbf{A}$  中每个矩阵  $\mathbf{A}_k$  的每行至多只有 2 个非零元素。

**约束 2：**限定  $\mathbf{A}$  中每个矩阵  $\mathbf{A}_k$  满足以下要求： $\mathbf{A}_k[l, m] \in x + jy$ ， $x, y \in \mathcal{P}$ ， $\mathcal{P} = 0, \pm 1, \pm 2, \dots, \pm 2^{(q-1)}$ ， $k = 1, 2, \dots, K$ ， $l, m = 1, 2, \dots, N$  其中， $\mathbf{A}_k[l, m]$  表示矩阵  $\mathbf{A}_k$  第  $l$  行第  $m$  列的元素。在问题 2,3,4 中，固定  $q = 3$ ；在问题 5 中，需要寻找合适的  $q$  以满足精度要求，并且使得硬件复杂度  $C$  尽量低。

**问题 1：**对于  $N = 2^t$ ， $t = 1, 2, 3, \dots$  的 DFT 矩阵  $\mathbf{F}_N$ ，在满足约束 1 的条件下，对最优问题中的变量  $\mathcal{A}$  和  $\beta$  进行优化，并计算最小误差和方案的硬件复杂度  $C$ 。

**问题 2：**讨论通过限制  $\mathbf{A}_k$  中元素实部和虚部取值范围的方式来减少硬件复杂度的方案。对于  $N = 2^t$ ， $t = 1, 2, 3, 4, 5$  的 DFT 矩阵  $\mathbf{F}_N$ ，在满足约束 2 的条件下，对  $\mathcal{A}$  和  $\beta$  进行优化，并计算最小误差和方案的硬件复杂度  $C$ 。

**问题 3：**同时限制  $\mathbf{A}_k$  的稀疏性和取值范围。对于  $N = 2^t$ ， $t = 1, 2, 3, 4, 5$  的 DFT 矩阵  $\mathbf{F}_N$ ，请在同时满足约束 1 和 2 的条件下，对  $\mathcal{A}$  和  $\beta$  进行优化，并计算最小误差和方案的硬件复杂度  $C$ 。

**问题 4：**考虑矩阵  $\mathbf{F}_N = \mathbf{F}_{N_1} \otimes \mathbf{F}_{N_2}$ ，其中  $\mathbf{F}_{N_1}$  和  $\mathbf{F}_{N_2}$  分别是  $N_1$  和  $N_2$  维的 DFT 矩阵。当  $N_1 = 4$ ， $N_2 = 8$  时，在同时满足约束 1 和 2 的条件下，对  $\mathcal{A}$  和  $\beta$  进行优化，并计算最小误差和方案的硬件复杂度  $C$ 。

**问题 5：**在问题 3 的基础上加上精度的限制来研究矩阵分解方案。要求将精度限制在 0.1 以内，即  $\text{RMSE} \leq 0.1$ 。对于  $N = 2^t$ ， $t = 1, 2, 3, \dots$  的 DFT 矩阵  $\mathbf{F}_N$ ，请在同时满足约束 1 和 2 的条件下，对  $\mathcal{A}, \beta, \mathcal{P}$  进行优化，并计算方案的硬件复杂度  $C$ 。

## 1.3 模型总览

我们针对以上五个问题进行了因果分析，如图 1.1 所示，问题 1 和问题 2 作为问题 3 的基础，而问题 4 和问题 5 是问题 3 的拓展。在下面的论文中将要指出的是我们分别从解析方向和优化算法两个方向求解矩阵分解逼近问题。第一问中，我们使用递归库利-图基快速傅里叶变换算法对任意  $N = 2^t$  维度的 DFT 矩阵做分解，在这种情况下 RMSE 为 0，并

## 模型总览

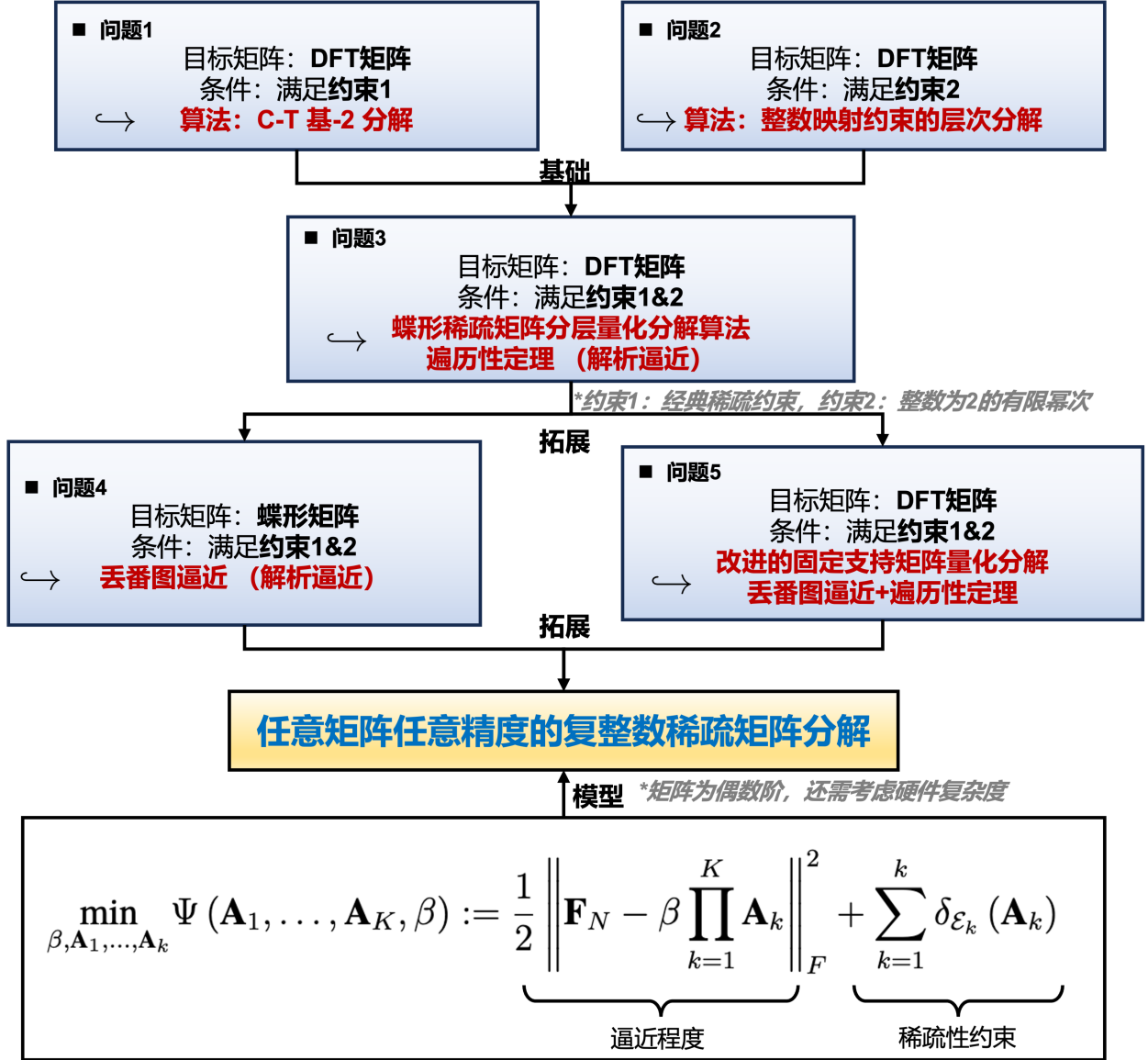


图 1.1 问题因果及本文建立模型总览

展示了它的硬件复杂度。第二问题中我们借助柔性近似多层稀疏变换算法，提出了整数映射约束的层次分解，该方法基于梯度方向对矩阵分解因子迭代优化，将矩阵分解并满足约束 2。第三问中，我们从解析的角度提出遍历性定理，从优化算法的角度提出蝶形稀疏分层量化分解算法都能将 DFT 矩阵分解为满足约束 1、2 的矩阵因子。在理论上能达到无限逼近原始矩阵。在第四问中，我们使用丢番图逼近算法直接对题目中给定的  $\mathbf{F}_n$  进行求解，给出了一个极为逼近的结果。问题 5 中，从优化方法的角度，我们提出改进的固定支持矩阵量化分解，使用遗传算法寻找矩阵因子元素的取值。从解析的角度，我们结合遍历性定理和丢番图逼近能够实现任意精度对任意 DFT 矩阵进行逼近。

## 2 模型的假设与定义

**假设 1** 我们假设矩阵缩放因子  $\beta > 0$ ，这是因为若  $\beta = 0$ ，可以通过设计  $\mathbf{A}_k = \mathbf{0}$  满足  $\text{RMSE} = 0$  以实现模糊放缩。在下面的章节中，我们将  $\beta$  作为估计矩阵的放缩系数。将问题重新叙述为：

$$\min_{\mathcal{A}, \beta} \text{RMSE}(\mathcal{A}, \beta) = \frac{1}{N} \|\mathbf{F}_N - \beta \prod_k^K \mathbf{A}_k\|_F \quad (2.5)$$

**定义 1** 我们所感兴趣的对象是  $DFT$  的矩阵结构，因此  $\mathbf{F}_N$  被重定义为  $\sqrt{N}\mathbf{F}_N$ 。即：

$$\mathbf{F}_N = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & w & w^2 & \cdots & w^{N-1} \\ 1 & w^2 & w^4 & \cdots & w^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w^{N-1} & w^{2(N-1)} & \cdots & w^{(N-1)(N-1)} \end{bmatrix} \quad (2.6)$$

事实上，通过设计  $\beta_0 = \sqrt{N}$  可以还原对题中所要求的  $\mathbf{F}_N$  逼近。

**定义 2 (乘法器的硬件复杂度)** 根据题意，对多个矩阵连乘  $\prod_k \mathbf{A}_k, \mathbf{A}_k \in \mathbb{C}^{n \times n}$  的硬件复杂度计算方法可以有多种计算方式：

- 方案 1：计算连续的两个矩阵之间的复乘次数并累加。对于  $K$  个连乘矩阵，共需要进行  $K - 1$  次计算。
- 方案 2：从最右端开始，计算  $\mathbf{A}_2 \cdot \mathbf{A}_1$  的复乘次数，然后计算他们的乘法结果  $\mathbf{A}_{21}$ ，再计算与  $\mathbf{A}_3$  的复乘次数。对于  $K$  个连乘矩阵，共需要进行  $K - 1$  次计算。

对于方案 1，可以通过在两个矩阵之间插入单位阵来规避复杂度，这与实际情况不符。因此，本文采用更为公平的方案 2 所定义的复乘次数计算方法。

若  $\beta$  不为 1，则乘法次数为对  $\mathbf{F}_N$  的逼近矩阵中不为  $\pm 1, \pm j, \pm 1 \pm j$  的元素个数。需要说明的是，尽管可以通过设计  $\beta_0 = \sqrt{N}$  逼近题中所定义的  $\mathbf{F}_N$ ，但我们仍然定义  $\beta$  的默认值为 1。题目中对于复数乘法的定义不清晰，根据附录一：名词解释中的解释，我们只有复数与复数乘法时计算复数乘法次数，实数与复数之间乘法不计算复数乘法次数。

## 3 符号说明

在本文中，矩阵采用加粗大写字母表示（例如： $\mathbf{A}$ ），向量用小写加粗字母表示（例如： $\mathbf{a}$ ），集合用花体字母表示（例如： $\mathcal{A}$ ）。连乘运算规则被定义为  $\prod_{i=k}^K \mathbf{A}_k = \mathbf{A}_K \cdots \mathbf{A}_1$ 。对于使用索引  $i, j$  访问矩阵的元素形式为  $a_{i,j} = \mathbf{A}(i, j)$ 。问题中一些预定义的重要符号列举在下方的表格中，其余符号和运算符则在文中给出了定义。

表 3.1 运算符号

| 运算符号                                                | 意义                                                  |
|-----------------------------------------------------|-----------------------------------------------------|
| $\otimes$                                           | 矩阵的 Kronecker 积                                     |
| $\ \cdot\ _0$                                       | 矩阵的 $\ell_0$ 范数                                     |
| $\ \cdot\ _2$                                       | 矩阵的谱半径                                              |
| $\ \cdot\ _F$                                       | Frobenius 范数                                        |
| $\text{Proj}(\cdot)$                                | 投影算子                                                |
| $\text{vec}(\cdot)$                                 | 标准向量化运算符                                            |
| $\text{diag}(i, j, k, \dots)$                       | 对角线元素为 $i, j, k, \dots$ 的对角矩阵                       |
| $\text{blkdiag}(\mathbf{A}_1, \mathbf{A}_2, \dots)$ | 对角线元素为矩阵 $\mathbf{A}_1, \mathbf{A}_2, \dots$ 的块对角矩阵 |
| $\text{supp}(\cdot) \in \mathbb{B}^{m \times n}$    | 矩阵零/非零元素构成的 0-1 矩阵                                  |
| $k : r : n - 1$                                     | 从 $k$ 开始以 $r$ 为间隔地访问元素，直到 $n - 1$                   |
| $\llbracket t \rrbracket$                           | $1, \dots, t$ 构成的集合                                 |
| $\text{tr}(\cdot)$                                  | 矩阵的迹                                                |

## 4 问题 1 的分析与求解

在本节中，我们首先指出稀疏矩阵逼近是一个 NP-hard 问题，这指导我们选用比优化逼近方法效果更好的分解方法。首先引入蝶形矩阵并将其实现重要的对角化分解。接着介绍 C-T 基 2 分解法的原理和算法，并给出以此法对 DFT 矩阵的稀疏矩阵逼近结果（部分）和硬件复杂度。

### 4.1 问题 1 分析

问题 1 要求待设计矩阵  $\mathbf{A}_k$  满足约束 1 但不限定  $\mathbf{A}_k$  元素的取值范围，以逼近  $N = 2^t$  规模的 DFT 矩阵，同时对最优化问题中的变量  $\mathcal{A}$  和  $\beta$  进行优化，并计算最小误差和方案的硬件复杂度。题目中的矩阵缩放因子  $\beta$  是为了避免当约束集（正）同质时自然产生的缩放模糊性，对因子进行归一化并在数据保真度项中引入的 [2]。DFT 的稀疏矩阵乘积逼近是一种明显快速的矩阵向量乘法，与任意向量相乘的成本将与乘积中矩阵的非零总数成正

表 3.2 一般符号

| 符号                                           | 说明                                                                       |
|----------------------------------------------|--------------------------------------------------------------------------|
| $N$                                          | DFT 矩阵的维度 (2 的次幂)                                                        |
| $K$                                          | 设计的矩阵 $\mathbf{A}_k$ 的个数                                                 |
| $C$                                          | 硬件复杂度                                                                    |
| $q$                                          | $\mathbf{A}_k$ 中元素的取值范围                                                  |
| $L$                                          | 复数乘法次数                                                                   |
| $\beta$                                      | 实值矩阵缩放因子                                                                 |
| $j$                                          | $j = \sqrt{-1}$ 表示虚数单位                                                   |
| $\mathbf{e}_i \in \mathbb{R}^n$              | 维度为 $n$ 的标准正交基的第 $i$ 个单位向量, $\mathbf{e}_i(i) = 1$ , 其余元素为 0              |
| $\mathbf{A}_k$                               | 待设计的第 $k$ 个矩阵                                                            |
| $\mathbf{A}_{t,k}$                           | $N = 2^t$ 的 DFT 矩阵库利-图基分解的第 $k$ 个矩阵                                      |
| $\mathbf{S}^{(\ell)}, \ell \in \mathbf{N}^+$ | 预设支持矩阵                                                                   |
| $\mathbf{I}$                                 | 单位矩阵                                                                     |
| $\mathbf{U}$                                 | 全部元素为 1 的矩阵                                                              |
| $\mathbf{F}_N$                               | $N$ 阶 DFT 矩阵                                                             |
| $\mathbf{D}_{x,y,z,\dots}$                   | 对角线元素为 $x, y, z, \dots$ 的对角矩阵                                            |
| $\mathcal{A}$                                | $\mathcal{A} = \mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k$ 待设计矩阵的集合 |
| $\mathcal{P}$                                | $\mathbf{A}_k$ 中元素实部与虚部的取值集合                                             |

比。不难发现, 问题 1 等价于下列稀疏矩阵逼近问题:

$$\min_{\mathcal{A}, \beta} \frac{1}{2} \|\mathbf{F}_N - \beta \prod_{k=1}^K \mathbf{A}_k\|_F^2 \quad (4.7)$$

其中, 若稀疏矩阵  $\mathbf{A}_K$  从特定的稀疏矩阵模板中选取, 则称具有固定支持约束 (fixed-support constraints)。支持 (support) 是对应于矩阵中非零元素的索引集, 并且是先验已知的。固定支持约束将每个  $\mathbf{A}_k$  限制为具有包含在给定预设支持  $\mathbf{S}^{(\ell)}, \ell \in \mathbf{N}^+$  中。我们将在下文中指



出，任意  $2^t$  阶蝴蝶结构矩阵可以被  $t$  个蝴蝶形式的稀疏矩阵模板（被称为蝴蝶因子）唯一分解。若稀疏矩阵不是预设的，则称其具有经典稀疏约束（即题给约束 1）。在全文中，我们提到稀疏矩阵时均默认其满足约束 1。业已证明，若  $\mathbf{A}_k$  包含经典稀疏矩阵约束甚至是固定支持约束，即使  $K = 2$ ，该问题也是 NP-hard 的 [3]。由于本题中  $\mathbf{A}_k$  的元素不受限制，随着问题规模  $N$  的指数增长，传统的梯度搜索方法难以在多项式时间内找到解，更难以讨论硬件复杂度的优化。

针对问题 1 难以优化搜索的特点，我们对问题 1 中要求的  $2^N$  次方规模 DFT 矩阵使用递归库利-图基快速傅里叶变换算法（recursive Cooley-Tukey Fast Fourier Transform, C-TFFT）分解。该迭代算法在面对偶数阶 DFT 矩阵时具有良好的性质，能够将 DFT 分解为  $\log_2(N)$  个稀疏矩阵和一个排列矩阵（permutation matrix）的乘积，并且多项式乘法次数是  $\mathcal{O}(N \log N)$  的。C-TFFT 的迭代计算方法决定了其在面对任意  $2^t$  维度 DFT 时计算快速，并且易于存储。我们将在问题 1 的求解中指出并对比，不经过硬件复杂度优化时，C-TFFT 算法的结果能够实现极低的硬件复杂度，同时由于算法是对 DFT 的精确分解，计算误差  $\text{RMSE} = 0$ ，远好于基于梯度下降的搜索算法（ $\text{RMSE} = 0.71 \pm 0.05$ ）。

## 4.2 递归库利-图基快速傅里叶变换算法

递归库利-图基快速傅里叶变换算法利用了傅里叶级数的周期性质，一些额外的块矩阵表示法使我们能够将库利-图基过程与 DFT 矩阵的稀疏因式分解联系起来 [4]。C-T 基-2 (radix-2) 分解法可以将任意  $N = 2^t$  维度的 DFT 矩阵  $\mathbf{F}_N$  使用与  $\mathbf{F}_{N/2}$  相关的矩阵表达，以迭代完成对  $N$  阶  $\mathbf{F}_N$  的计算，即著名的 [C-T 基-2 分解](#)。分解后的矩阵表示是借助对蝶形矩阵的乘积（被称为蝶形操作，butterfly operator）实现的。本节中的蝶形矩阵实际上属于一类 [蝶形因子](#)（butterfly factor），他的优美特性是我们的求解方案的基础，我们将在问题 3 的解决方案中给出详细的定义。为了避免混淆，将本小节中构造的  $\mathbf{B}_N$  称为蝶形矩阵，并且再次强调它是一类蝶形因子。

### 4.2.1 蝶形矩阵

对于  $N = 2^t$ ，定义基-2 蝶形矩阵  $\mathbf{B}_N \in \mathbb{C}^{N \times N}$  为  $\mathbf{B}_N = \begin{bmatrix} \mathbf{I}_{N_*} & \boldsymbol{\Omega}_{N_*} \\ \mathbf{I}_{N_*} & -\boldsymbol{\Omega}_{N_*} \end{bmatrix}$ ，其中  $\boldsymbol{\Omega}_{N_*} = \text{diag}(1, \omega_t, \dots, \omega_t^{N_*-1})$ ， $\omega_t = \exp(-2\pi j/N)$ 。  $t$  级的 DFT 可以由  $t-1$  级的 DFT 表示为

$$\mathbf{F}_N \mathbf{x}(k : r : n-1) = \mathbf{B}_N \begin{bmatrix} \mathbf{F}_{N_*} \mathbf{x}(k : r_* : n-1) \\ \mathbf{F}_{N_*} \mathbf{x}(k + r : r_* : n-1) \end{bmatrix} \quad (4.8)$$

其中  $r = n/N$ ,  $N_* = N/2$ ,  $r_* = 2r$ 。对于  $n = N$ ，我们有  $\mathbf{F}_n \mathbf{x} = \mathbf{B}_n \begin{bmatrix} \mathbf{F}_{n/2} \mathbf{x}(0 : 2 : n-1) \\ \mathbf{F}_{n/2} \mathbf{x}(1 : 2 : n-1) \end{bmatrix}$ 。

递归结果表明， $\mathbf{F}_n$  可以按照以下形式分解：

$$\mathbf{F}_n \mathbf{x} = \mathbf{A}_t \dots \mathbf{A}_1 \mathbf{P}_n^\top \mathbf{x} \quad (4.9)$$



其中  $\mathbf{P}_n$  是对  $\mathbf{x}$  的某种排列，被称为排列矩阵。 $\mathbf{A}_t$  是蝴蝶矩阵的块对角，例如

$$\mathbf{A}_t = \text{blkdiag}(\underbrace{\mathbf{B}_N, \dots, \mathbf{B}_N}_r) = \mathbf{I}_r \otimes \mathbf{B}_N \quad N = 2^t, r = n/L \quad (4.10)$$

关于蝶形矩阵  $\mathbf{B}_N$ ，还将用到他的一个重要性质。注意到  $\mathbf{B}_N$  还可以表示为

$$\mathbf{B}_N = \begin{bmatrix} \mathbf{I}_{N_*} & \Omega_{N_*} \\ \mathbf{I}_{N_*} & -\Omega_{N_*} \end{bmatrix} = \mathbf{P}_\Lambda \Lambda \triangleq \begin{bmatrix} \mathbf{I}_{N_*} & \mathbf{I}_{N_*} \\ \mathbf{I}_{N_*} & -\mathbf{I}_{N_*} \end{bmatrix} \begin{bmatrix} \mathbf{I}_{N_*} & \\ & \Omega_{N_*} \end{bmatrix} \quad (4.11)$$

其中  $\Lambda_N$  为对角阵。结合 Kronecker 乘子  $\otimes$  的性质，我们有

$$\mathbf{A}_t = \mathbf{I}_r \otimes \mathbf{B}_N = \mathbf{I}_r \otimes (\mathbf{P}_{\Lambda_N} \Lambda_N) = (\mathbf{I}_r \otimes \mathbf{P}_{\Lambda_N})(\mathbf{I}_r \otimes \Lambda_N) \quad (4.12)$$

需要指出的是，上式将 DFT 的分解矩阵  $\mathbf{A}_t$  中可能出现的无理数部分分离为对角阵  $\mathbf{I}_r \otimes \Lambda_N$ ，则对  $\mathbf{A}_t$  的整数逼近可以被极大地简化为对对角阵  $\Lambda_N$  的逼近。设  $N$  阶  $\Lambda$  的矩阵乘法逼近为  $\Lambda \approx \hat{\Lambda} = \prod_i \hat{\Lambda}_i$ ，随之产生新的逼近矩阵  $\mathbf{A}_i = \mathbf{I}_r \otimes \hat{\Lambda}_i$ 。我们将在问题 3 中充分利用该性质。

#### 4.2.2 C-T 基-2 分解

公式 (4.9) 表明存在对  $\mathbf{F}_N, N = 2^t, t \in \mathbb{N}^+$  的如下形式分解：

$$\mathbf{F}_N \mathbf{P}_N = (\mathbf{I}_1 \otimes \mathbf{B}_N)(\mathbf{I}_2 \otimes \mathbf{B}_{N/2}) \dots (\mathbf{I}_{N/2} \otimes \mathbf{B}_2)$$

关于排列矩阵  $\mathbf{P}_N$ ，定义  $\Pi_N = \mathbf{I}_N(:, v), v = [0:2:n-1, 1:2:n-1]$ ，有如下引理：

**引理 1 [5]** 设  $N = 2^t$ ， $\mathbf{P}_N$  定义为  $\mathbf{P}_N = \Pi_N(\mathbf{I}_2 \otimes \mathbf{P}_{N/2})$ ，且  $\mathbf{R}_q = \mathbf{I}_{2^{t-q}} \otimes \Pi_{2^q}, q \in \llbracket t \rrbracket$ ，那么  $\mathbf{P}_N = \mathbf{R}_t \dots \mathbf{R}_1$ 。

注意到  $\mathbf{P}_N^{-1} = \mathbf{P}_N^\top$ ，利用引理 1，可以预计算出给定维度  $N$  的 DFT 矩阵的位反转排列  $\mathbf{P}_N$ ，我们将在问题 3 的求解中利用该性质。下面给出 C-T 基-2 分解定理：

**定理 1 (C-T 基-2 分解定理)** 若  $N = 2^t$ ，那么

$$\mathbf{F}_N = \mathbf{A}_t \dots \mathbf{A}_1 \mathbf{P}_N^\top$$

其中  $\mathbf{P}_N$  如引理 1 所定义，并且有：

$$\begin{aligned} \mathbf{A}_q &= \mathbf{I}_r \otimes \mathbf{B}_N, & L &= 2^q, r = N/L, \\ \mathbf{B}_L &= \begin{bmatrix} \mathbf{I}_{L_*} & \Omega_{L_*} \\ \mathbf{I}_{L_*} & -\Omega_{L_*} \end{bmatrix}, & L_* &= L/2, \\ \Omega_{L_*} &= \text{diag}(1, \omega_L, \dots, \omega_L^{L_*-1}), & \omega_L &= \exp(-2\pi j/L) \end{aligned} \quad (4.13)$$

该定理的证明略。 $\mathbf{F}_N$  的 C-T 基-2 分解是稀疏分解, 并且得到的矩阵  $\mathbf{A}_k$  均满足稀疏矩阵约束 1。下面将要看到, C-T 基-2 分解法实际上是问题 3 中提出的[蝶形稀疏矩阵分层量化分解](#)的平凡结果。需要再次声明的是, 我们所分解的对象不包括原题中定义的 DFT 矩阵的常数项  $\frac{1}{\sqrt{N}}$ , 但可以将其设置为  $\beta$ 。

碍于篇幅限制, 验证了对  $N = 2^t, t = 2, 3$  阶 DFT 矩阵应用 C-T 基-2 分解法的分解结果, 得到的稀疏矩阵逼近展开为:

- 4 阶 DFT 矩阵,  $N = 2^2 = 4, \omega = \exp(-2\pi j/4) = -j$ :

$$\mathbf{F}_4 = \mathbf{A}_2 \mathbf{A}_1 \mathbf{P}_4^\top = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & \omega \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -\omega \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_3 & \mathbf{e}_2 & \mathbf{e}_4 \end{bmatrix} \quad (4.14)$$

- 8 阶 DFT 矩阵,  $N = 2^3 = 8, \omega = \exp(-2\pi j/8) = \frac{1}{\sqrt{2}}(1 - 1j), \omega^2 = -j, \omega^3 = -\frac{1}{\sqrt{2}}(1 + j)$ :

$$\begin{aligned} \mathbf{F}_8 &= \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1 \mathbf{P}_8^\top \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & \omega & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & \omega^3 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -\omega & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -\omega^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -\omega^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & \omega^2 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -\omega^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & \omega^2 \\ 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & -\omega^2 \end{bmatrix} \\ &\times \left( \mathbf{I}_4 \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_5 & \mathbf{e}_3 & \mathbf{e}_7 & \mathbf{e}_2 & \mathbf{e}_6 & \mathbf{e}_4 & \mathbf{e}_8 \end{bmatrix} \end{aligned} \quad (4.15)$$

- $2^t$  阶 DFT 矩阵, 设  $N = 2^t, t = 2, 3, \dots$ , 令  $q = 1, 2, \dots, t$ , 应用 C-T 基-2 分解方法 (定理 1) 计算  $\mathbf{A}_q$  和利用引理 1 计算  $\mathbf{P}_N$ , 则  $\mathbf{F}_N = \prod_{q=1}^t \mathbf{A}_q \mathbf{P}_N^\top$ 。

### 4.3 矩阵逼近结果评估

#### 4.3.1 Frobenius 范数误差

由于 C-T 基-2 分解是基于对  $\mathbf{F}_N$  的迭代计算, 在不限元素类型时, 分解得到的矩阵对 DFT 矩阵的误差 Frobenius 范数 (公式 2.5) 为 0。

#### 4.3.2 硬件复杂度

使用 C-T 基-2 分解在定义 2 下的硬件复杂度如表 4.3 所示

表 4.3 C-T 基-2 分解的硬件复杂度表

| 阶数 ( $N$ )    | 2 | 4 | 8 | 16  | 32   | 64    | 128    | 256    | 512     | 1024     | ... |
|---------------|---|---|---|-----|------|-------|--------|--------|---------|----------|-----|
| 硬件复杂度 ( $C$ ) | 0 | 0 | 0 | 512 | 5120 | 32768 | 172032 | 811008 | 3588096 | 15269888 | ... |

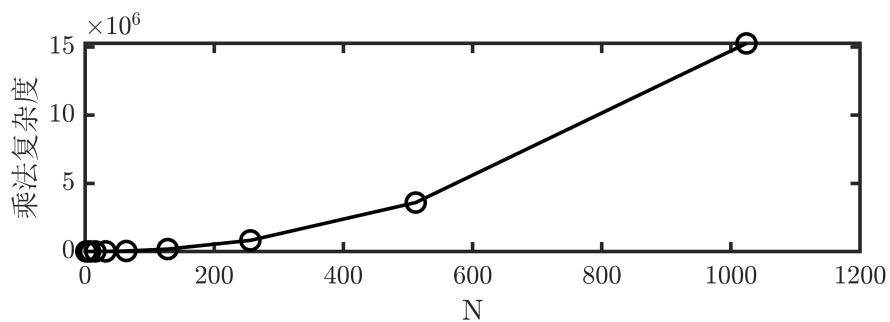


图 4.2 C-T 基-2 分解的硬件复杂度

更直观地，给出  $t$  级（阶数为  $2^t$ ）的 DFT 的结果如图 4.2 所示。C-T 基-2 分解方法在分解  $\mathbf{F}_4$  和  $\mathbf{F}_8$  时具有误差为 0，硬件复杂度为 0 的优点。而随着  $t$  的增长，模型阶数呈指数式增长，硬件复杂度也呈现指数式增长。

## 5 问题 2 分析与求解

### 5.1 问题 2 分析

问题 2 中的  $\mathbf{A}_k$  需满足约束 2，即要求复数域内的整数分解，具体来说， $\mathbf{A}_k$  中的元素需要满足

$$\mathbf{A}_k[l, m] \in \{x + jy \mid x, y \in \mathcal{P}\}, \mathcal{P} = \{0, \pm 1, \pm 2, \dots, \pm 2^{q-1}\}, k = 1, 2, \dots, K; l, m = 1, 2, \dots, N$$

其中  $q = 3$ 。

我们采用了[整数映射约束的层次分解](#)方法来解决这个问题。该方法基于柔性近似多层稀疏变换 (Flexible Approximate Multi-layer Sparse Transforms) 和层次分解 (Hierarchical Factorization)，目的是将一个任意矩阵  $A$  分解为若干个稀疏矩阵的乘积，该方法能对分解的稀疏矩阵做约束。整数映射约束的层次分解的主要思想是在柔性近似多层稀疏变换算法的迭代过程中仅限制元素取值，这样得出来的矩阵分解就满足约束 2。

### 5.2 整数映射约束的层次分解

考虑一个矩阵  $\mathbf{F}_N \in \mathbb{R}^{n \times n}$ ，我们的目标函数是找到一组稀疏矩阵因式  $\mathbf{A}_k \in \mathbb{R}^{n \times n}, k \in \{1 \dots K\}$  使得  $\mathbf{F}_N \approx \prod_{j=1}^J \mathbf{A}_k$ ，并且满足约束。该目标函数由下表示 [6]：

$$\min_{\beta, \mathbf{A}_1, \dots, \mathbf{A}_K} \Psi(\mathbf{A}_1, \dots, \mathbf{A}_K, \beta) := \frac{1}{2} \left\| \mathbf{F}_N - \beta \prod_{k=1}^K \mathbf{A}_k \right\|_F^2 + \sum_{k=1}^K \delta_{\mathcal{E}_k}(\mathbf{A}_k). \quad (5.16)$$

其中  $\delta_{\mathcal{E}_k}(\cdot)$  是约束集合  $\mathcal{E}_k$  的指标函数，在目标函数中作为惩罚项，即：若  $S_k$  满足约束，则  $\delta_{\mathcal{E}_k}(S_k) = 0$ ，反之  $\delta_{\mathcal{E}_k}(S_k) = 1$ 。虽然问题 2 未对分解的矩阵的稀疏性做要求，我们可以松弛稀疏性约束，但为了算法介绍的完整性一并介绍。

#### 5.2.1 柔性近似多层稀疏变换

公式 (5.16) 是高度非凸的，导致稀疏性的惩罚通常是非光滑的。为了处理这个目标函数，Bolte[7] 提出了一种称为[邻近交替线性化最小化](#)(Proximal Alternating Linearized Minimization, PALM) 的算法，通过近端梯度步骤交替更新每个矩阵因式。文献 [7] 中的目标函数为

$$\Phi(\mathbf{x}_1, \dots, \mathbf{x}_N) := H(\mathbf{x}_1, \dots, \mathbf{x}_N) + \sum_{j=1}^N f_j(\mathbf{x}_j) \quad (5.17)$$

PALM 方法的简略的伪代码为：

其中  $Proj_{\mathcal{T}_j}(\cdot)$  是在集合  $\mathcal{T}_j$  上的投影算子， $c_j^i$  定义了步长，取决于  $H$  的梯度的 Lipschitz 常数。利用 PALM，可直接求解问题 (5.16)。

---

**算法 1 PALM 算法**


---

```

1: for  $i = 1, \dots, Niter$  do
2:   for  $j = 1, \dots, N$  do
3:      $\mathbf{x}_j^{i+1} \leftarrow Proj_{\mathcal{T}_j} \left( \mathbf{x}_j^i - \frac{1}{c_j^i} \nabla_{\mathbf{x}_j} H(\mathbf{x}_1^{i+1} \dots \mathbf{x}_j^i \dots \mathbf{x}_N^i) \right)$ 

```

---

### 1. 投影算子

PALM 依赖于每次迭代时每个因子在约束集上的投影，因此投影算子应该简单且易于计算。这里介绍我们使用的整数映射方法，我们借鉴了神经网络模型量化的思想，提出了最近映射，非对称映射和对称映射三种方法。

(a) 最近映射  $Proj_{nearest}(\cdot)$ : 设  $S$  是一个在实数或者高维空间中的数的集合。给定任意一个数或点  $x$ ，我们想要找到一个  $s \in S$ ，使得  $s$  是离  $x$  最近的点，即：

$$s \leftarrow \arg \min_{y \in S} d(x, y) \quad (5.18)$$

这里  $d(x, y)$  是  $x$  和  $y$  之间的欧式距离，对于实数他就是绝对值，对于二维空间或高维空间，他是欧氏距离公式。

(b) 非对称映射  $Proj_{asym}(\cdot)$ : 如图5.3(a)所示，对于区间  $[a, b]$  中的某个值  $x$ ，其在区间  $[c, d]$  中的对应值  $y$  可以通过以下公式计算：

$$y = c + \frac{(x - a)(d - c)}{b - a} \quad (5.19)$$

为了将一个矩阵  $M$  中的每一个元素  $m_{ij}$  从  $[a, b]$  映射到  $[c, d]$ ，我们可以使用上述公式为每一个元素执行这个变换。

$$m'_{ij} = c + \frac{(m_{ij} - a)(d - c)}{b - a} \quad (5.20)$$

其中， $m'_{ij}$  是变换后的元素值。

(c) 对称映射  $Proj_{sym}(\cdot)$ : 如图5.3(b)所示，该映射原理与非对称映射相似，即将初始被映射区间设定为  $[-b, b]$ 。

在后续的实验中我们发现使用最近映射将矩阵因子的元素限制在约束 2 中效果最好，我们只报告最近映射的结果。

### 2. 算法原理

我们指定 PALM 用于多层稀疏逼近问题的迭代次数为  $i$ ，分解的矩阵因子个数  $K$ ， $S_k^i$  是第  $i$  次迭代中的第  $k$  个矩阵因式。定义  $\mathbf{L} := \prod_{\ell=k+1}^K \mathbf{S}_\ell^i$  意义为  $S_\ell^i$  左边的矩阵乘积；定义  $\mathbf{R} := \prod_{\ell=1}^{k-1} \mathbf{S}_\ell^{i+1}$  意义为  $S_\ell^i$  右边的矩阵乘积，若没有则表示为单位矩阵： $\prod_{\ell \in \emptyset} \mathbf{S}_\ell = \mathbf{Id}$ 。我们使用不带稀疏惩罚项的目标函数

$$H(\mathbf{A}_1^{k+1}, \dots, \mathbf{A}_{k-1}^{i+1}, \mathbf{A}_k^i, \dots, \mathbf{A}_K^i, \beta^i) = H(\mathbf{L}, \mathbf{A}_k^i, \mathbf{R}, \beta^i) = \frac{1}{2} \|\mathbf{F}_N - \beta^i \mathbf{L} \mathbf{A}_k^i \mathbf{R}\|_F^2 \quad (5.21)$$

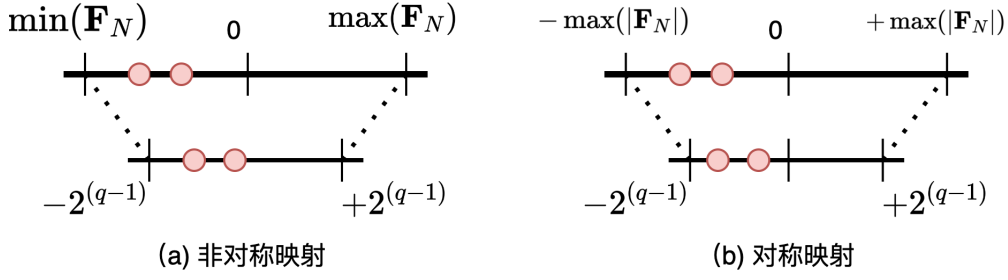


图 5.3 两种投影算子

的梯度

$$\nabla_{\mathbf{A}_k^i} H(\mathbf{L}, \mathbf{A}_k^i, \mathbf{R}, \beta^i) = \beta^i \mathbf{L}^T (\beta^i \mathbf{L} \mathbf{A}_k^i \mathbf{R} - \mathbf{F}_N) \mathbf{R}^T \quad (5.22)$$

来更新  $\mathbf{S}_k^i$ 。当第  $i$  次的所有因式都更新完一遍后，再将  $\beta$  更新。使用  $\hat{\mathbf{F}}_N = \prod_{k=1}^K \mathbf{A}_k^{i+1}$  来表示  $i+1$  轮的估计值，上式 (5.22) 可以写为：

$$H(\mathbf{A}_1^{i+1}, \dots, \mathbf{A}_K^{i+1}, \beta^i) = \frac{1}{2} \|\mathbf{F}_N - \beta^i \hat{\mathbf{F}}_N\|_F^2 \quad (5.23)$$

式 (5.23) 对  $\beta^i$  的梯度为：

$$\nabla_{\beta^i} H(\mathbf{A}_1^{i+1}, \dots, \mathbf{A}_K^{i+1}, \beta^i) = \beta^i \text{Tr}(\hat{\mathbf{F}}_N^T \hat{\mathbf{F}}_N) - \text{Tr}(\mathbf{F}_N^T \hat{\mathbf{F}}_N) \quad (5.24)$$

### 3. 算法的初始化设置

算法默认的初始化值为  $\beta^0 = 1$ ，对于第一个矩阵设置为零矩阵，即  $\mathbf{A}_1^0 = \mathbf{0}$ 。对于  $k \geq 2$  的矩阵因子设置为单位阵，即  $\mathbf{A}_k^0 = \mathbf{I}_d$ 。在代码实现中我们将步长定义为： $c_k^i = (1 + \alpha) \cdot (\beta^i)^2 \|\mathbf{R}\|_2^2 \cdot \|\mathbf{L}\|_2^2$ ，其中  $\alpha = 1e^{-3}$ 。对于 DFT 矩阵该分解为多少个因式矩阵  $K$ ，根据文献 [8] 和第 6.3 章的表述，任意  $N \times N$  ( $N = 2^t$ ,  $t = 2, 3, 4, 5, \dots$ ) 维的矩阵可以用  $\log_2(N)$  个个数的无约束的蝴蝶矩阵完美逼近。因此，我们对于  $\mathbf{F}_N$  的 DFT 矩阵，将  $K$  设定为  $\log_2(N)$ 。除此之外，该算法的收敛性对  $\{\mathbf{A}_k^0\}_{k=1}^K$  的初始值非常敏感，因此我们采用蝴蝶矩阵作为柔性近似多层稀疏变换算法的预设矩阵模板（即初始值）。例如  $\mathbf{F}_8$  预设的三个蝴蝶矩阵为：

$$\mathbf{S}_{bf}^{(1:1)} = \begin{bmatrix} 1.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 1.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 1.0 \end{bmatrix} \quad (5.25)$$

$$\mathbf{S}_{bf}^{(2:2)} = \begin{bmatrix} 1.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 1.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 1.0 \end{bmatrix} \quad (5.26)$$

$$\mathbf{S}_{bf}^{(3:3)} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix} \quad (5.27)$$

蝴蝶矩阵的生成方法由公式 (6.48) 给出。

柔性近似多层稀疏变换的过程如伪代码 2 所示：

---

**算法 2** [柔性近似多层稀疏变换算法\(palm4msa\)](#)

---

**输入：** 需要分解的矩阵  $\mathbf{F}_N$ ；被分解的个数  $K$ ；初始因式矩阵模板（初始值） $\{\mathbf{A}_k^0\}_{k=1}^K$ ； $\beta^0 = 1$ ；迭代次数  $N$ 。

**输出：** 分解的  $K$  个矩阵因式  $\{\mathbf{A}_k^N\}_{k=1}^K$ ；实值矩阵缩放因子  $\beta^N$ 。

```

1: for  $i = 0, \dots, N - 1$  do
2:   for  $k = 1, \dots, K$  do
3:      $\mathbf{L} \leftarrow \prod_{\ell=k+1}^K \mathbf{A}_\ell^i$ 
4:      $\mathbf{R} \leftarrow \prod_{\ell=1}^{k-1} \mathbf{A}_\ell^{i+1}$ 
5:     Set  $c_k^i > (\beta^i)^2 \|\mathbf{R}\|_2^2 \cdot \|\mathbf{L}\|_2^2$ 
6:      $\mathbf{A}_k^{i+1} \leftarrow Proj_{\{nearest, asym, sym\}} \left( \mathbf{A}_k^i - \frac{1}{c_k^i} \beta^i \mathbf{L}^T (\beta \mathbf{L} \mathbf{A}_k^i \mathbf{R} - \mathbf{F}_N) \mathbf{R}^T \right)$  # 满足约束 2
7:    $\hat{\mathbf{F}}_N \leftarrow \prod_{k=1}^K \mathbf{A}_k^{i+1}$ 
8:    $\beta^{i+1} \leftarrow \frac{\text{Tr}(\mathbf{F}_N^T \hat{\mathbf{F}}_N)}{\text{Tr}(\hat{\mathbf{F}}_N^T \hat{\mathbf{F}}_N)}$ 

```

---

整数映射约束的层次分解算法通过整数映射对矩阵因式 (第 6 行) 交替更新，步长由梯



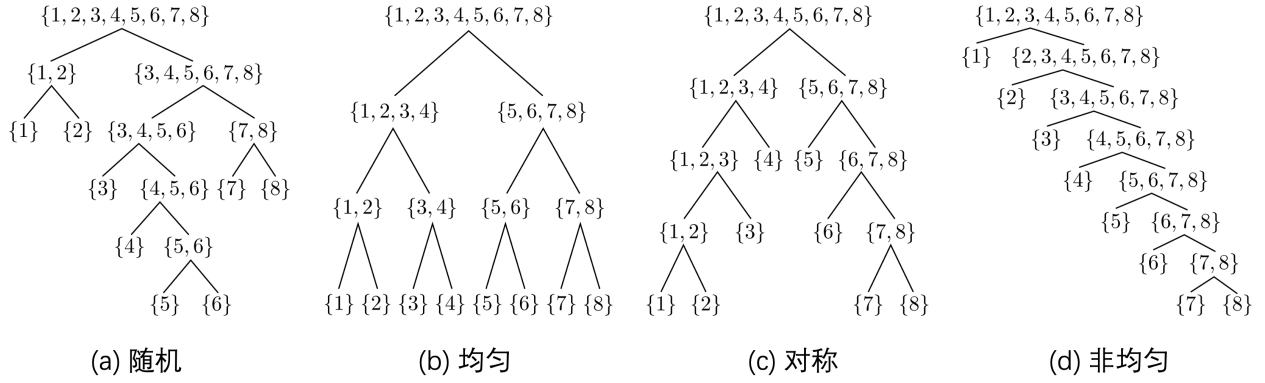


图 5.4 层次分解的顺序

度的 Lipschitz 模控制 (第 5 行), 在每一轮迭代后更新  $\beta$  (第 8 行), 最后得到收敛后的  $J$  个矩阵因式  $\{\mathbf{A}_k^N\}_{k=1}^K$ 。

### 5.2.2 层次分解

通过实验注意到, 对于整数映射约束的层次分解算法, 每次设定更少的因子分解数量  $K$  可以获得更好的近似结果。因此我们提出层次分解的方法, 每次只使用柔性近似多层稀疏变换对矩阵进行二分分解。

实际上, 待分解的矩阵  $\mathbf{F}_N = \prod_{k=1}^K \mathbf{A}_k$  可以先分解为  $\mathbf{F}_N = \mathbf{T}_1 \mathbf{A}_1$ , 其中  $\mathbf{T}_1 = \prod_{k=2}^K \mathbf{A}_k$ 。该整数映射约束的层次分解算法迭代地将输入矩阵  $\mathbf{F}_N$  分解为 2 个因子, 一个是稀疏的 (对应于  $\mathbf{A}_1$ ), 另一个是不稀疏的 (对应于  $\mathbf{T}_1$ )。在稀疏度较低的因子  $\mathbf{T}_1$  上重复该过程, 直到获得所需的因子个数  $K$ 。在每一步, 可以对目前引入的所有因子进行全局优化, 以使所有的矩阵因子乘积逼近原始矩阵  $\mathbf{F}_N$ 。

由于我们每次将矩阵一分为二, 因此分解方法以及初始模板分配就会有很多种结构 (如图 5.4 所示): 初始因式矩阵模板的蝶式矩阵设置也需要与层次分解的顺序一致。在整数映射约束的层次分解算法中, 我们使用非均匀的分解形式。

该算法的伪代码如下所述, 具体来说,  $\mathbf{F}_N$  矩阵要被二分为  $K$  个因子, 需要进行  $K-1$  次迭代。每一次先将不稀疏矩阵使用 palm4msa 算法二分为两个稀疏矩阵 (第 3 行), default\_init 的意思是使用预设的蝶式模板矩阵和初始的  $\beta'$ ; 再将得到的稀疏矩阵作为下一个 palm4msa 算法的初始值输入进去得到  $\ell+1$  次分解 (第 6 行), current\_init 的意思是使用前一次分解的矩阵作为初始模板矩阵,  $\beta = \beta\beta'$ 。经过  $K-1$  次迭代后我们就能得到精度更高的满足约束 2 的整数矩阵分解。

### 5.3 问题 2 求解

对于  $N = 2$  的情况, DFT 矩阵不用分解, 并且满足约束 2。我们分别对  $N = 2^t$ ,  $t = 2, 3, 4, 5$  的 DFT 矩阵  $\mathbf{F}_N$  直接使用整数映射约束的层次分解算法, 结果展示在下表中。

---

**算法 3** 整数映射约束的层次分解算法


---

**输入:** 需要分解的矩阵  $\mathbf{F}_N$ ; 被分解的个数  $K$ ; 初始因式矩阵模板 (初始值)  $\{\mathbf{A}_k^0\}_{k=1}^J$ ;  $\beta^0 = 1$ ;  $\ell \in \{1 \dots K-1\}$ 。

**输出:** 分解的  $K$  个矩阵因式  $\{\mathbf{A}_k\}_{k=1}^K$ ; 实值矩阵缩放因子  $\beta^N$ 。

- 1:  $\mathbf{T}_0 \leftarrow \mathbf{F}_N$
- 2: **for**  $\ell = 0, \dots, K-1$  **do**
- 3:      $\beta', \{\mathbf{A}_2, \mathbf{A}_1\} = \text{palm4msa}(\mathbf{T}_{\ell-1}, \quad 2, \text{init} = \text{default\_init})$  # 先将  $\mathbf{T}_0$  分解为两个模板
- 4:      $\mathbf{T}_\ell \leftarrow \lambda' \mathbf{A}_2$
- 5:      $\mathbf{S}_\ell \leftarrow \mathbf{A}_1$
- 6:      $\beta, \{\mathbf{T}_\ell, \{\mathbf{A}_k\}_{k=1}^\ell\} = \text{palm4msa}(\mathbf{T}_{\ell-1}, \quad \ell+1, \text{init} = \text{current\_init})$  # 在用第一次分解得到的矩阵作为模板再一次分解

- 7:  $\mathbf{A}_J \leftarrow \mathbf{T}_{K-1}$

---

**表 5.4 问题二结果**

| 序号 | $\mathbf{F}_N$    | 分解个数 | 映射方法    | RMSE     | 硬件复杂度 | $\beta$ |
|----|-------------------|------|---------|----------|-------|---------|
| 1  | $\mathbf{F}_2$    | 1    |         | 0        | 0     | 1       |
| 2  | $\mathbf{F}_4$    | 2    | nearest | 0.780624 | 0     | 0.0625  |
| 3  | $\mathbf{F}_8$    | 3    | nearest | 0.873269 | 0     | 0.1222  |
| 4  | $\mathbf{F}_{16}$ | 4    | nearest | 0.93729  | 0     | 0.0617  |
| 5  | $\mathbf{F}_{32}$ | 5    | nearest | 0.96872  | 0     | 0.0310  |

1.  $\mathbf{F}_4 = \beta \mathbf{A}_2 \mathbf{A}_1$

其中

$$\mathbf{A}_2 = \begin{bmatrix} (1+0j) & 0j & (1+0j) & 0j \\ 0j & (1+0j) & 0j & (1+0j) \\ (1+0j) & (-1+0j) & (1+0j) & (-1+0j) \\ 0j & (1+0j) & 0j & (1+0j) \end{bmatrix} \quad (5.28)$$

$$\mathbf{A}_1 = \begin{bmatrix} (1+0j) & (1+0j) & 0j & 0j \\ (1+0j) & (1+0j) & (-1+0j) & 0j \\ 0j & 0j & (1+0j) & (1+0j) \\ 0j & 0j & 0j & (1+0j) \end{bmatrix} \quad (5.29)$$

2.  $\mathbf{F}_8 = \beta \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1$

其中

$$\mathbf{A}_3 = \begin{bmatrix} (1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j \\ 0j & 0j & (1+0j) & 0j & 0j & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j & (1+0j) \\ (1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j \\ 0j & 0j & (1+0j) & 0j & 0j & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j & (1+0j) \end{bmatrix} \quad (5.30)$$

$$\mathbf{A}_2 = \begin{bmatrix} (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j \\ (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & (1+0j) & 0j & (1+0j) \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & (1+0j) & 0j & (1+0j) \end{bmatrix} \quad (5.31)$$

$$\mathbf{A}_1 = \begin{bmatrix} (2+0j) & (1+0j) & 0j & 0j & (2+0j) & 0j & 0j & 0j \\ (2+0j) & (1+0j) & 0j & 0j & (-2+0j) & 0j & 0j & 0j \\ (2+0j) & 0j & (1+0j) & (1+0j) & (2+0j) & 0j & 0j & 0j \\ (2+0j) & 0j & (1+0j) & (1+0j) & (-2+0j) & 0j & 0j & 0j \\ (2+0j) & 0j & 0j & 0j & (2+0j) & (1+0j) & 0j & 0j \\ (2+0j) & 0j & 0j & 0j & (-1+0j) & (1+0j) & 0j & 0j \\ (2+0j) & 0j & 0j & 0j & (2+0j) & 0j & (1+0j) & (1+0j) \\ (2+0j) & 0j & 0j & 0j & (-2+0j) & 0j & (1+0j) & (1+0j) \end{bmatrix} \quad (5.32)$$

3.  $\mathbf{F}_{16} = \beta \mathbf{A}_4 \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1$

限于篇幅，矩阵  $A$  放在[附录 A](#)。

4.  $\mathbf{F}_{32} = \beta \mathbf{A}_5 \mathbf{A}_4 \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1$

限于篇幅，矩阵  $A$  放在[附录 A](#)。

## 6 问题 3 分析与求解

### 6.1 问题 3 分析

问题 3 要求待设计矩阵  $\mathbf{A}_k$  同时满足约束 1 和约束 2，同时最优化  $\mathcal{A}, \beta$ 。鉴于问题 1 和问题 2 的结果，自然想到找到一种方法，将满足约束 1 产生的无精度损失的矩阵中包含无理数的因子逼近，从而实现最小化误差的目标。为此，我们针对问题 3 提出了两种算法，分别是基于遍历性定理的逼近方法和蝶形稀疏矩阵分层量化分解方法。

对于前者，我们试图使用解析方法得到精确度较高的解，对于后者，我们介绍一种基于固定支持稀疏矩阵的分解方法，该方法依赖于蝶形结构矩阵分解的唯一性定理。结合设计的 $\beta$ -最优贪心策略，可以对任意  $N = 2^t$  维度的  $\mathbf{F}_N$  实现逼近。并且指出问题 1 中所使用的 C-T 基-2 分解方法是本方法在不需要满足约束 2 时的一个特例。本方法可以在计算复杂度为 0 时实现较高精度的逼近。有趣的是，我们指出允许反复应用稀疏蝶形矩阵层次量化分解方法产生大量稀疏矩阵逼近目标矩阵，且目标矩阵也可以不再被要求是蝶形结构矩阵。该部分内容将在 6.3 节中介绍。

### 6.2 $\mathbf{F}_8$ 的结构分析与求解

对于  $t = 3$  的情况，

$$\mathbf{F}_8 = \mathbf{A}_{3,3}\mathbf{A}_{3,2}\mathbf{A}_{3,1}\mathbf{P}_8^\top \quad (6.33)$$

其中

$$\mathbf{P}_8^\top = [\mathbf{e}_1, \mathbf{e}_5, \mathbf{e}_3, \mathbf{e}_7, \mathbf{e}_2, \mathbf{e}_6, \mathbf{e}_4, \mathbf{e}_8] \quad (6.34)$$

$$\mathbf{A}_{3,1} = \mathbf{I}_4 \otimes \mathbf{B}_2 = \mathbf{I}_4 \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (6.35)$$

$$\mathbf{A}_{3,2} = \mathbf{I}_2 \otimes \mathbf{B}_4 = \mathbf{I}_2 \otimes \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -j \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & j \end{bmatrix} \quad (6.36)$$

都满足约束 2，且没有误差。只有

$$\mathbf{A}_{3,3} = \begin{bmatrix} \mathbf{I}_4 & \Omega_4 \\ \mathbf{I}_4 & -\Omega_4 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_4 & \mathbf{I}_4 \\ \mathbf{I}_4 & -\mathbf{I}_4 \end{bmatrix} \begin{bmatrix} \mathbf{I}_4 & \mathbf{0} \\ \mathbf{0} & \Omega_4 \end{bmatrix} \quad (6.37)$$

不直接满足约束 2。其中

$$\begin{bmatrix} \mathbf{I}_4 & \mathbf{0} \\ \mathbf{0} & \Omega_4 \end{bmatrix} = \text{diag}(1, 1, 1, 1, 1, w_8, w_8^2, w_8^3) \quad (6.38a)$$

$$= \text{diag}(1, 1, 1, 1, 1, \frac{\sqrt{2}}{2}(1-j), -j, -\frac{\sqrt{2}}{2}(1+j)) \quad (6.38b)$$

$$= \frac{1}{\sqrt{2}} \text{diag}(1, 1, 1, 1, 1, 1-j, -j, -1-j) \text{diag}(\sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}, 1, \sqrt{2}, 1) \quad (6.38c)$$

$$= \text{diag}(1, 1, 1, 1, 1, 1, -j, -j) \text{diag}(1, 1, 1, 1, 1, \frac{\sqrt{2}}{2}(1-j), 1, \frac{\sqrt{2}}{2}(1-j)) \quad (6.38d)$$

式 (6.38c) 的第二个对角矩阵的所有元素具有相同的方向，但是长度不同，式 (6.38d) 的第二个对角矩阵的所有元素具有相同的长度，但是方向不同。针对长度不同方向相同的对角矩阵，本文提出采用[丢番图逼近](#)（见问题 4 定理3）的方法来近似，针对长度相同方向不同的对角矩阵，本文提出采用[遍历性定理](#)的方法来逼近。

### 6.2.1 遍历性分析

式 (6.38d) 的第二个对角矩阵  $\text{diag}(1, 1, 1, 1, 1, -j, -j) \text{diag}(1, 1, 1, 1, 1, \frac{\sqrt{2}}{2}(1-j), 1, \frac{\sqrt{2}}{2}(1-j))$  共有两个不同元素 1 和  $\frac{\sqrt{2}}{2}(1-j)$ ，其长度相等，角度分别为 0 和  $-\frac{\pi}{4}$ ，相差  $\pi/4$ ，为此我们选定一个  $2 \times 2$  的起始对角矩阵  $\mathbf{S}$ ，然后使用  $2 \times 2$  对角矩阵  $\mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3, \dots$  对  $\mathbf{S}$  进行旋转以获得需要的角度差。由于起始对角矩阵  $\mathbf{S}$  的两个元素必须具有相同长度，旋转矩阵  $\mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3, \dots$  的两个元素也必须具有相同长度，由图6.5可见，起始对角矩阵  $\mathbf{S}$  的两个元素的可选组合共有 10 个：

$$(1, 1), (j, 1), (-1, 1), (-j, 1),$$

其角度分别为

$$(0, 0), (\frac{\pi}{2}, 0), (\pi, 0), (\frac{3\pi}{2}, 0)$$

以及

$$(2+j, 2-j), (1+2j, 2+j), (-1+2j, 2-j)$$

其角度分别为

$$(\arctan \frac{1}{2}, -\arctan \frac{1}{2}), (\arctan 2, \arctan \frac{1}{2}), (\frac{\pi}{2} + \arctan \frac{1}{2}, -\arctan \frac{1}{2})$$

以及

$$(4+j, 4-j), (1+4j, 4+j), (-1+4j, 4-j)$$

其角度分别为

$$(\arctan \frac{1}{4}, -\arctan \frac{1}{4}), (\arctan 4, \arctan \frac{1}{4}), (\frac{\pi}{2} + \arctan \frac{1}{4}, -\arctan \frac{1}{4})$$

在选择旋转矩阵之前，我们需要考虑哪些角度差是可以被旋转到的。为此我们需要

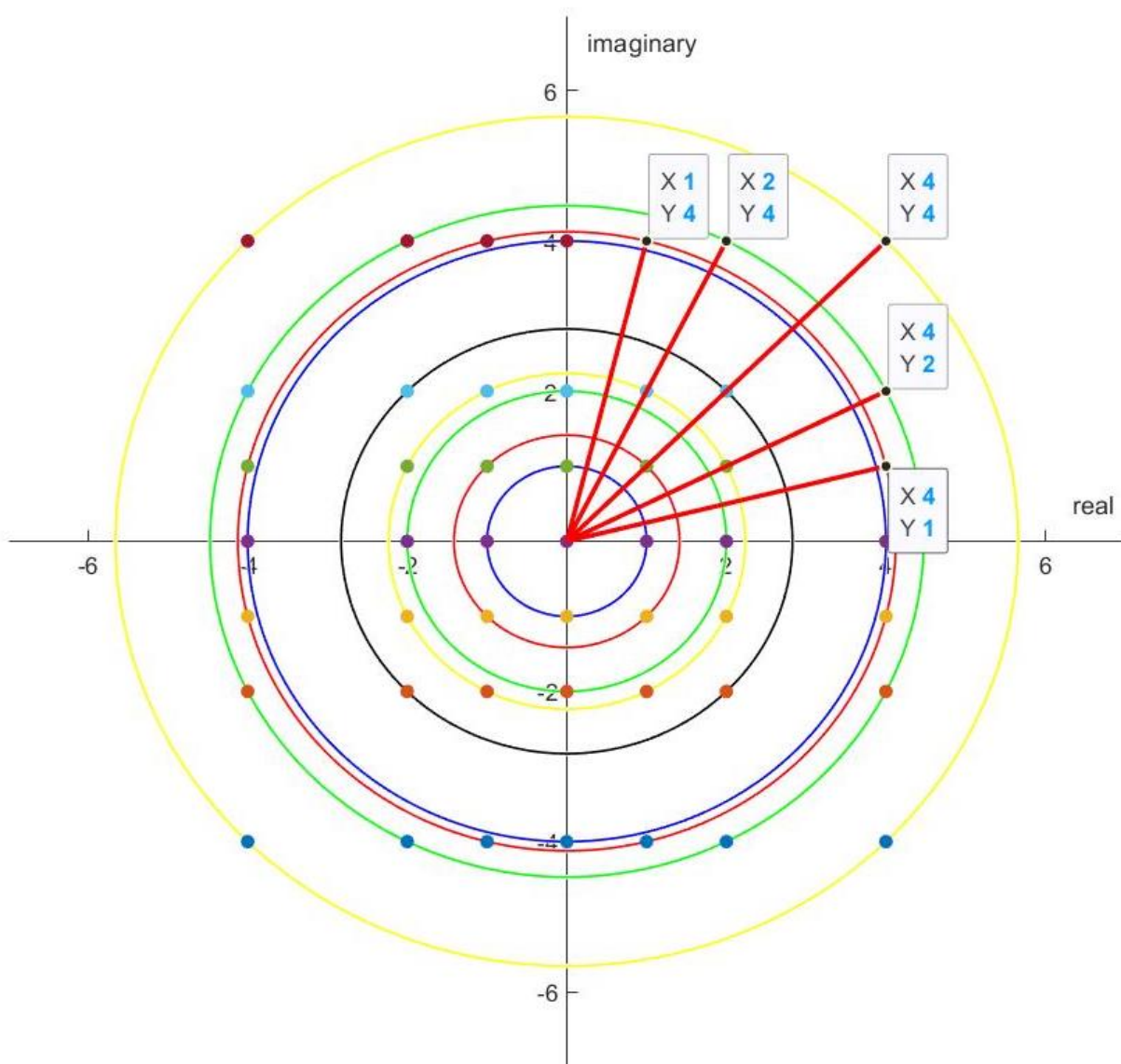


图 6.5 满足约束 2 的复平面上的元素

**引理 2 (Weyl 均匀分布定理)** 当  $\alpha \in (0, 1)$  是无理数时, 序列

$$\{n \cdot 2\pi\alpha \bmod 2\pi \mid n \in \mathbb{N}_+\} = 2\pi\alpha, 4\pi\alpha, 8\pi\alpha \dots \bmod 2\pi \quad (6.39)$$

在圆周  $\mathbb{R}/2\pi\mathbb{Z} = [0, 2\pi]$  上均匀分布。令映射  $S : [0, 2\pi] \rightarrow [0, 2\pi]$  表示转动  $2\pi\alpha$  弧度, 其定义为  $Sy = y + 2\pi\alpha \bmod 2\pi$ , 那么  $S$  是遍历的。

考虑圆环面  $Y = [0, 2\pi] \times [0, 2\pi]$ 。对于无理数  $\alpha, \beta \in (0, 1)$ , 令  $R$  表示转动  $2\pi\alpha$  弧度,  $S$  表示转动  $2\pi\beta$  弧度。  $R$  和  $S$  都是遍历的。令映射  $T : Y \rightarrow Y$  定义为  $T(x, y) = (Rx, Sy)$ 。那么  $T$  对于乘积测度是遍历的当且仅当  $\alpha$  和  $\beta$  对于有理系数是线性无关的。

如果  $\alpha$  和  $\beta$  对于有理系数是线性无关的, 那么两个角度的轨迹在  $[0, 2\pi] \times [0, 2\pi]$  上是稠密的, 我们就可以将起始矩阵  $\mathbf{S}$  的两个元素以任意精度旋转到任意的角度组合。旋转矩阵的

可选组合有 4 个

$(2 + j, 2 - j)$ , 其角度为  $(\arctan \frac{1}{2}, -\arctan \frac{1}{2})$      $(1 + 2j, 2 + j)$ , 其角度为  $(\arctan 2, \arctan \frac{1}{2})$

$(4 + j, 4 - j)$ , 其角度为  $(\arctan \frac{1}{4}, -\arctan \frac{1}{4})$      $(1 + 4j, 4 + j)$ , 其角度为  $(\arctan 4, \arctan \frac{1}{4})$

由于

$$\arctan \frac{1}{2} - \arctan \frac{1}{2} = 0, \arctan 2 + \arctan \frac{1}{2} = \frac{\pi}{2}, \arctan \frac{1}{4} - \arctan \frac{1}{4} = 0, \arctan 4 + \arctan \frac{1}{4} = \frac{\pi}{2}$$

这些角度组合除以  $2\pi$  之后对于有理系数是线性相关的, 根据 Weyl 均匀分布定理, 旋转矩阵能够到达的角度不是遍历的, 这意味我们可能无法以任意精度逼近想要的角度差。但是我们仍然可以尝试使得误差最小化。

因为我们可以使用  $\pm j$  和  $-1$  在不改变长度的情况下转动  $\frac{\pi}{2}$  的整数倍, 所以对于  $\mathbf{F}_8$  我们的目标角度差是

$$\frac{\pi}{4}, \frac{3\pi}{4}, \frac{5\pi}{4}, \frac{7\pi}{4} \mod 2\pi$$

中的任意一个。实际角度差与目标角度差的误差的最小值记为角度差误差 ( $E_{diff}$ )。

最后, 起始矩阵  $\mathbf{S}$  在经过旋转矩阵的旋转之后必须达到我们需要的角度 ( $0$  和  $-\frac{\pi}{4}$ ), 只要最后对应于 1 的元素被旋转到任何满足约束 2 的复数的角度附近, 我们就可以通过一个调整矩阵  $\mathbf{T}$  将其旋转回正实数轴。由于  $q = 3$ , 目标角度可以取的值有

$$0, \arctan \frac{1}{4}, \arctan \frac{1}{2}, \frac{\pi}{4}, \arctan 2, \frac{\pi}{2} \mod \frac{\pi}{2}$$

中的任意一个。实际旋转后角度与目标角度的误差的最小值记为目标角度误差 ( $E_{target}$ )。

整体算法如下: 给定最大的旋转矩阵数量 ( $n$ ), 遍历  $\mathbf{S}$  的 10 个可选组合和旋转矩阵的 4 个可选组合, 总误差 ( $E_{total}$ ) = 角度差误差 ( $E_{diff}$ ) + 目标角度误差 ( $E_{target}$ ), 取总误差最小的组合。

---

#### 算法 4 遍历性搜索算法

---

**输入:** 最大的旋转矩阵数量  $n$ ; 起始矩阵  $\mathbf{S}$ ; 旋转矩阵  $\mathbf{R}$

**输出:** 总误差最小的组合  $\mathbf{S}^*, \mathbf{R}^*$ 。

- 1: **for**  $\mathbf{S}' \in \mathbf{S}$  **do**
  - 2:     **for**  $\mathbf{R}' \in \mathbf{R}$  **do**
  - 3:         Compute  $E_{total} = E_{diff} + E_{target}$
  - 4:  $\{\mathbf{S}^*, \mathbf{R}^*\} \leftarrow \operatorname{argmin} E_{total}$
-



### 6.2.2 遍历性定理逼近求解

对于  $\mathbf{F}_8$ ，我们的目标是逼近  $\text{diag}(1, \frac{\sqrt{2}}{2}(1-j))$ ，结果如表6.5所示。式 (6.41) 对应于表6.5的第一列  $n = 2$ ，其起始矩阵  $\mathbf{S} = \text{diag}(1+4j, 4+j)$ ，旋转矩阵

$$\mathbf{R}_1 \mathbf{R}_2 = [\text{diag}(1+2j, 2+j)]^2 \quad (6.40)$$

调整矩阵  $\mathbf{T} = \text{diag}(-2+j, -1-2j)$ 。表6.5表明，目标角度误差 ( $E_{target}$ ) 在  $n = 2$  时达到最小值，随着旋转矩阵数量的增多，角度差误差 ( $E_{diff}$ ) 稳定下降，目标角度误差则起伏波动，这说明两个角度的旋转轨迹在  $E_{diff} = 0$  处是稠密的，然而  $E_{target} = 0$  的某个邻域位于旋转轨迹之外，目标角度误差无法小于某个正数（即使使用全部 4 个旋转矩阵的组合仍然无法使目标角度误差继续下降，已通过随机仿真证明，此处略去）。因此再增加旋转矩阵对于精度的提升非常有限，我们选择表6.5的第一列  $n = 2$  的结果来逼近  $\mathbf{F}_8$ 。

表 6.5  $\text{diag}(1, \frac{\sqrt{2}}{2}(1-j))$  的逼近精度表

| 旋转矩阵数量 ( $n$ )          | 2        | 20        | 71        | 146       | 622                     | ... |
|-------------------------|----------|-----------|-----------|-----------|-------------------------|-----|
| 总误差 ( $E_{total}$ )     | 0.076772 | 0.075054  | 0.074154  | 0.071537  | 0.070982                | ... |
| 角度差误差 ( $E_{diff}$ )    | 0.011647 | 0.0082107 | 0.0064118 | 0.0011769 | $6.7174 \times 10^{-5}$ | ... |
| 目标角度误差 ( $E_{target}$ ) | 0.065125 | 0.066843  | 0.067743  | 0.07036   | 0.070915                | ... |

$$\begin{aligned} & \frac{1}{5\sqrt{85}} \begin{bmatrix} -2+j & 0 \\ 0 & -1-2j \end{bmatrix} \begin{bmatrix} 1+4j & 0 \\ 0 & 4+j \end{bmatrix} \begin{bmatrix} 1+2j & 0 \\ 0 & 2+j \end{bmatrix}^2 \\ & \approx \begin{bmatrix} 0.9979 - 0.0651j & 0 \\ 0 & 0.6508 - 0.7592j \end{bmatrix} \approx \begin{bmatrix} 1 & 0 \\ 0 & \frac{\sqrt{2}}{2}(1-j) \end{bmatrix} \end{aligned} \quad (6.41)$$

而6.38d的第二个对角矩阵逼近式为

$$\begin{aligned} & \text{diag}(1, 1, 1, 1, 1, \frac{\sqrt{2}}{2}(1-j), 1, \frac{\sqrt{2}}{2}(1-j)) \\ & \approx \beta_8 \mathbf{D}_1 \mathbf{D}_2 \mathbf{D}_3^2 \quad (\beta_8 = \frac{1}{5\sqrt{85}}, \mathbf{D}_1, \mathbf{D}_2, \mathbf{D}_3 \text{ 是下面 3 行的对角矩阵}) \\ & = \frac{1}{5\sqrt{85}} \text{diag}(-2+j, -2+j, -2+j, -2+j, -2+j, -1-2j, -2+j, -1-2j) \\ & \times \text{diag}(1+4j, 1+4j, 1+4j, 1+4j, 1+4j, 4+j, 1+4j, 4+j) \\ & \times [\text{diag}(1+2j, 1+2j, 1+2j, 1+2j, 1+2j, 2+j, 1+2j, 2+j)]^2 \\ & \approx \text{blkdiag}((0.9979 - 0.0651j)\mathbf{I}_5, 0.6508 - 0.7592j, 0.9979 - 0.0651j, 0.6508 - 0.7592j) \end{aligned} \quad (6.42)$$



$$\begin{bmatrix} \hat{\mathbf{I}}_4 & \mathbf{0} \\ \mathbf{0} & \hat{\Omega}_4 \end{bmatrix} = \beta_8 \cdot \text{diag}(1, 1, 1, 1, 1, 1, -j, -j) \mathbf{D}_1 \mathbf{D}_2 \mathbf{D}_3^2 \quad (6.43)$$

因此

$$\hat{\mathbf{A}}_{3,3} = \begin{bmatrix} \mathbf{I}_4 & \mathbf{I}_4 \\ \mathbf{I}_4 & -\mathbf{I}_4 \end{bmatrix} \begin{bmatrix} \hat{\mathbf{I}}_4 & \mathbf{0} \\ \mathbf{0} & \hat{\Omega}_4 \end{bmatrix}, \quad \text{RMSE}(\hat{\mathbf{A}}_{3,3}) = 0.0341 \quad (6.44)$$

$$\begin{aligned} \hat{\mathbf{F}}_8 &= \hat{\mathbf{A}}_{3,3} \mathbf{A}_{3,2} \mathbf{A}_{3,1} \mathbf{P}_8^\top \\ &= \beta_8 \begin{bmatrix} \mathbf{I}_4 & \mathbf{I}_4 \\ \mathbf{I}_4 & -\mathbf{I}_4 \end{bmatrix} \text{diag}(1, 1, 1, 1, 1, 1, -j, -j) \mathbf{D}_1 \mathbf{D}_2 \mathbf{D}_3^2 \mathbf{A}_{3,2} \mathbf{A}_{3,1} \mathbf{P}_8^\top \\ &= \begin{bmatrix} 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j & 1.00 - 0.07j \\ 1.00 - 0.07j & 0.65 - 0.76j & -0.07 - 1.00j & -0.76 - 0.65j & -1.00 + 0.07j & -0.65 + 0.76j & 0.07 + 1.00j & 0.76 + 0.65j \\ 1.00 - 0.07j & -0.07 - 1.00j & -1.00 + 0.07j & 0.07 + 1.00j & 1.00 - 0.07j & -0.07 - 1.00j & -1.00 + 0.07j & 0.07 + 1.00j \\ 1.00 - 0.07j & -0.76 - 0.65j & 0.07 + 1.00j & 0.65 - 0.76j & -1.00 + 0.07j & 0.76 + 0.65j & -0.07 - 1.00j & -0.65 + 0.76j \\ 1.00 - 0.07j & -1.00 + 0.07j & 1.00 - 0.07j & -1.00 + 0.07j & 1.00 - 0.07j & -1.00 + 0.07j & 1.00 - 0.07j & -1.00 + 0.07j \\ 1.00 - 0.07j & -0.65 + 0.76j & -0.07 - 1.00j & 0.76 + 0.65j & -1.00 + 0.07j & 0.65 - 0.76j & 0.07 + 1.00j & -0.76 - 0.65j \\ 1.00 - 0.07j & 0.07 + 1.00j & -1.00 + 0.07j & -0.07 - 1.00j & 1.00 - 0.07j & 0.07 + 1.00j & -1.00 + 0.07j & -0.07 - 1.00j \\ 1.00 - 0.07j & 0.76 + 0.65j & 0.07 + 1.00j & -0.65 + 0.76j & -1.00 + 0.07j & -0.76 - 0.65j & -0.07 - 1.00j & 0.65 - 0.76j \end{bmatrix} \end{aligned} \quad (6.45)$$

$$\text{RMSE}(\hat{\mathbf{F}}_8) = \frac{1}{8} \|\mathbf{F}_8 - \hat{\mathbf{F}}_8\|_F = 0.0682, \quad \beta_8 = \frac{1}{5\sqrt{85}} \quad (6.46)$$

复杂度为  $L = 168$ ,  $C = 3 \times L = 504$ 。

### 6.3 蝶形稀疏矩阵分层量化分解

本小节介绍我们提出的一种**蝶形稀疏矩阵分层量化分解**方法，分解的结果均为预设支持的稀疏矩阵（蝶形因子，butterfly factor）。方法的有效性是基于以下事实：(1) 用蝶形结构组成矩阵可以精确地逼近任何给定的矩阵 [4]，(2)  $\mathbf{F}_N \mathbf{P}_N$  具有蝶形结构（butterfly structure），而蝶形结构矩阵又具有唯一的蝶形因子分解 [8]。我们将首先介绍蝶形因子，并指出问题 1 的 C-T 基-2 分解法中的  $\mathbf{B}_n$  是一类蝶形结构。然后介绍蝶形结构矩阵的逼近方法，并使用问题 2 中的最邻近投影方法使其元素的类型为规定值，同时设计了**最优- $\beta$  贪心策略**。

#### 6.3.1 蝶形因子与蝶形结构

**定义 3 (蝶形支持)**  $N = 2^t$  维的蝶形支持是  $t$  支撑元组  $\mathbf{S}_{bf} \triangleq (\mathbf{S}_{bf}^{(1)}, \dots, \mathbf{S}_{bf}^{(t)}) \in (\mathbb{B}^{N \times N})^t$ ，定义为

$$\mathbf{S}_{bf}^{(\ell)} \triangleq \mathbf{I}_{2^{\ell-1}} \otimes \mathbf{U}_{2 \times 2} \otimes \mathbf{I}_{N/2^\ell}, 1 \leq \ell \leq t \quad (6.47)$$

根据定义，问题 1 中的蝶形矩阵  $\mathbf{B}_N$  属于一类蝶形因子。此外，还需要定义蝶形结构

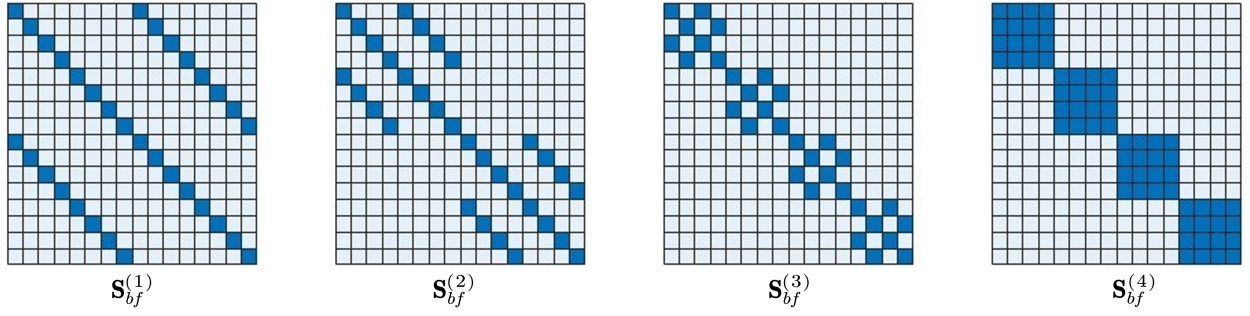


图 6.6 蝶形因子

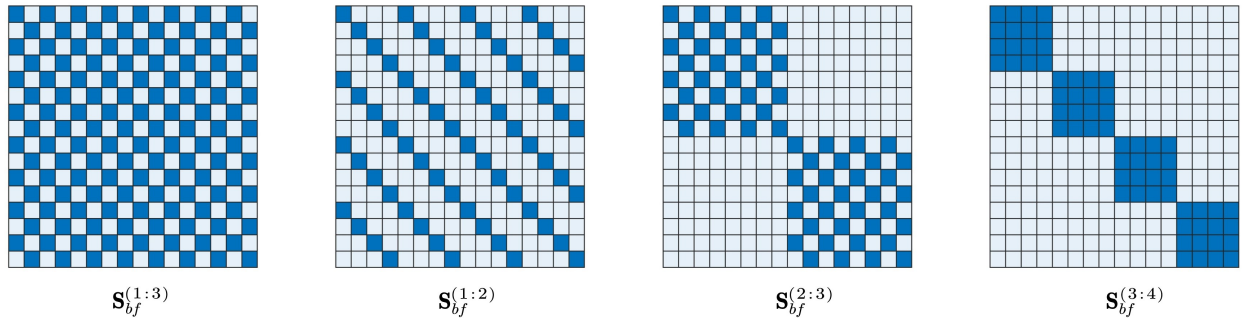


图 6.7 蝶形结构矩阵

**定义 4 (蝶形结构)** 称  $N = 2^t$  维的矩阵  $\mathbf{Z}$  具有蝶形结构, 如果他可以被分解为  $t$  个因子  $(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(t)})$ , 并具有蝶形支持  $\mathbf{S}_{bf} \triangleq (\mathbf{S}_{bf}^{(1)}, \dots, \mathbf{S}_{bf}^{(t)})$ , 满足

$$\mathbf{Z} = \mathbf{X}^{(1)} \dots \mathbf{X}^{(t)}$$

其中  $(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(t)}) \in \Sigma_{\mathbf{S}_{bf}^{(1)}} \times \dots \times \Sigma_{\mathbf{S}_{bf}^{(t)}}$ , 表明这些因子具备蝶形支持。

对于任何  $(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(t)}) \in \Sigma_{\mathbf{S}_{bf}^{(1)}} \times \dots \times \Sigma_{\mathbf{S}_{bf}^{(t)}}$ , 连续因子的部分积  $\mathbf{X}^{(1)} \dots \mathbf{X}^{(t)}$  ( $1 \leq p \leq q \leq t$ ) 可以被支持  $\mathbf{S}_{bf}^{(p:q)}$  所表达。具体表达形式为  $\text{supp}(\mathbf{X}^{(p)} \dots \mathbf{X}^{(q)}) \subseteq \mathbf{S}_{bf}^{(p:q)}$ , 其中

$$\begin{aligned} \mathbf{S}_{bf}^{(p:q)} &:= \mathbf{I}_{2^{p-1}} \otimes \mathbf{V}^{(p,q)} \in \mathbb{B}^{N \times N} \\ \mathbf{V}^{(p,q)} &:= \mathbf{U}_{2^{q-p+1}} \otimes \mathbf{I}_{N/2^q} \in \mathbb{B}^{\frac{N}{2^{p-1}} \times \frac{N}{2^{q-1}}} \end{aligned} \quad (6.48)$$

令  $N = 16$ , 蝶形因子和蝶形结构矩阵分别如图6.6和图6.7所示。

### 6.3.2 算法与求解

业已证明, 对于  $2^t$  阶的具备蝶形结构的矩阵, 可以被唯一分解为  $t$  个蝶形因子的乘积 [8]。基于该理论, 我们提出了[蝶形稀疏矩阵分层量化分解](#)算法, 用于求解以下优化问题:

$$\min_{\beta, \mathbf{A}_1, \dots, \mathbf{A}_k} = \frac{1}{2} \left\| \mathbf{F} - \beta \prod_{k=1}^t \mathbf{A}_k \right\|_F^2. \quad (6.49)$$

其中  $\mathbf{A}_k$  满足稀疏约束 1 的预设蝴蝶支持矩阵,  $\mathbf{A}_k$  的元素满足约束 2。若  $\mathbf{F}$  是满足蝴蝶结构的矩阵, 则分解方法是唯一的。不难发现,  $\mathbf{F}_N \mathbf{P}_N$  正具备蝴蝶结构, 在元素不受限时可以被完美地分解, 且分解方式唯一。在满足约束 2 时, 则需要对分解方式做出改进, 我们提出了[最优- \$\beta\$  贪心策略](#), 其核心思想是对不满足约束 2 的元素投影至满足约束 2 的集合内, 最后优化  $\beta$  使得目标函数最小。对于一个规模为  $N = 2^t$  的蝴蝶结构矩阵, 需要将其分为  $t$  个具备蝴蝶支持  $\mathbf{S}_{bf}^{(p)}, 1 \leq p \leq t$  的矩阵。采用与问题 2 求解时相同的分层计算思路, 首先选择一颗分类二叉树  $\mathcal{T}$  的节点  $\ell$ , 尝试求解二元分解问题  $\mathbf{F} = \mathbf{X}\mathbf{Y}^\top$ , 其中  $\mathbf{X}, \mathbf{Y}$  是满足蝴蝶结构  $\text{supp}(\mathbf{X}) \subseteq \Sigma_{\mathbf{S}_{bf}^{(1:\ell)}}, \text{supp}(\mathbf{Y}) \subseteq \Sigma_{\mathbf{S}_{bf}^{(\ell+1:t)}}$  的因子, 然后根据二叉树的节点继续对产生的中间矩阵  $\mathbf{X}, \mathbf{Y}$  做二元分解, 直到得到  $t$  个因子。该二元分解算法被称为固定支持矩阵量化分解 (Fixed-support Matrix Quantified Factorization, FSMQF), 其伪代码展示为:

---

**算法 5** 固定支持矩阵量化分解

---

**输出:** FSMQF( $\mathbf{Z} \in \mathbb{C}^{m \times n}, \mathbf{S}^L \in \mathbb{B}^{m \times r}, \mathbf{S}^R \in \mathbb{B}^{n \times r}$ )

- 1:  $(\mathbf{S}^i)_{i=1}^N \leftarrow \varphi(\mathbf{S}^L, \mathbf{S}^R)$
  - 2: **for**  $i = 1 \in \llbracket r \rrbracket$  **do**
  - 3:      $(\mathbf{x}_i, \mathbf{y}_i) \leftarrow \text{argmin}\{\frac{1}{2}\|\mathbf{Z} \odot \mathbf{S}^i - \mathbf{x}_i \mathbf{y}_i^\top\|_F^2\}$ , 满足  $\text{supp}(\mathbf{x}_i \mathbf{y}_i^\top) \subseteq \mathbf{S}^i$
  - 4:  $\mathbf{X} \leftarrow \text{Proj}(\mathbf{x}_1 \dots, \mathbf{x}_r) \in \mathbb{C}^{m \times r}$
  - 5:  $\mathbf{Y} \leftarrow \text{Proj}(\mathbf{y}_1 \dots, \mathbf{y}_r) \in \mathbb{C}^{n \times r}$
- 

代码中,  $\varphi(\mathbf{X}, \mathbf{Y}) \triangleq (\mathbf{X}_i \mathbf{Y}_i^\top)_{i=1}^r \in (\mathbb{C}^{m \times n})^r$  表示计算两个矩阵  $\mathbf{X}, \mathbf{Y}$  的张量分解。 $\mathbf{X} \odot \mathbf{S}$  表示将  $\mathbf{X}$  中与  $\mathbf{S}$  进行交运算, 使得  $\mathbf{X}$  具有与  $\mathbf{S}$  相同的支持。

算法中还存在两处自由度较大的设计:

1. 对于逐元素投影  $\text{Proj}(\cdot)$ , 可以采用问题 2 中提到的最邻近方法, 对称或非对称映射。对比问题 2 中的方法, 该方法的优势在于逼近结果可能采用全部的约束元素, 这是因为  $\mathbf{X}, \mathbf{Y}$  矩阵没有归一化要求。
2. 对于优化问题  $\text{argmin}\{\frac{1}{2}\|\mathbf{Z} \odot \mathbf{S}^i - \mathbf{x}_i \mathbf{y}_i^\top\|_F^2\}$ , 可以采用梯度下降或贪心算法进行处理。
  - (a) 对于梯度下降, 考虑目标函数对  $\mathbf{x}_i$  和  $\mathbf{y}_i$  的结果分别为:

$$\nabla_{\mathbf{x}_i} = (\mathbf{x}_i \mathbf{y}_i^\top - \mathbf{Z} \odot \mathbf{S}^i) \mathbf{y}_i, \quad \nabla_{\mathbf{y}_i} = (\mathbf{x}_i \mathbf{y}_i^\top - \mathbf{Z} \odot \mathbf{S}^i)^\top \mathbf{x}_i$$

并应用梯度下降算法  $\mathbf{x}_i \leftarrow \mathbf{x}_i - \lambda \nabla_{\mathbf{x}_i}, \mathbf{y}_i \leftarrow \mathbf{y}_i - \lambda \nabla_{\mathbf{y}_i}$  求解。需要注意因为  $\mathbf{x}_i, \mathbf{y}_i$  为零时带来的梯度消失, 可以通过引入摄动等方式解决。

- (b) 对于贪心算法, 则考虑对  $\mathbf{Z} \odot \mathbf{S}^i$  的 SVD 分解 [3]:  $\mathbf{Z} \odot \mathbf{S}^i = \sum \mathbf{u}_i \sigma_i \mathbf{v}_i^\top$ , 并取其奇异值贡献最大的  $\mathbf{u}_i \rightarrow \mathbf{x}_i$ , 和  $\sigma_i \mathbf{v}_i \rightarrow \mathbf{y}_i$ 。

基于选择的二叉树反复应用算法5，可以得到蝶形稀疏矩阵分层量化分解算法（Butterfly Sparse Matrix Hierarchical Factorization, BSMHF）：使用算法6遍历二叉树，即可获得量

---

**算法 6** 蝶形稀疏矩阵分层量化分解算法（将中间矩阵  $\mathbf{M}^{(p:q)}$  ( $1 \leq p \leq q \leq t$ ) 按树结构分解为两块。）

---

**输出:**  $\text{BSMHF}(\mathbf{M}^{(p:q)} \in \mathbb{C}^{m \times n}, \text{二叉树 } \mathcal{T}^{(p:q)} : \{p, q\})$

**输出:**  $(\hat{\mathbf{X}}^{(p)}, \dots, \hat{\mathbf{X}}^{(\ell)}, \hat{\mathbf{X}}^{(\ell+1)}, \dots, \hat{\mathbf{X}}^{(q)})$

- 1: **if**  $p = q$  **then**
  - 2:     输出  $\mathbf{M}^{(p:q)}$
  - 3:  $\ell \leftarrow$  选择  $\mathcal{T}^{(p:q)}$  的最左子节点（如图5.4所示）
  - 4:  $(\mathcal{T}^{(p:\ell)}, \mathcal{T}^{(\ell+1:q)}) \leftarrow$  更新  $\mathcal{T}^{(p:q)}$  的子树
  - 5:  $(\mathbf{S}_{bf}^{(p:\ell)}, \mathbf{S}_{bf}^{(\ell+1:q)}) \leftarrow$  计算左, 右支持（公式6.48）
  - 6:  $(\mathbf{M}^{(p:\ell)}, \mathbf{M}^{(\ell+1:q)})^\top \leftarrow \text{FSMQF}(\mathbf{M}^{(p:q)}, \mathbf{S}_{bf}^{(p:\ell)}, \mathbf{S}_{bf}^{(\ell+1:q)})^\top$ （算法5）
  - 7:  $(\hat{\mathbf{X}}^{(p)}, \dots, \hat{\mathbf{X}}^{(\ell)}) \leftarrow \text{BSMHF}(\mathbf{M}^{(p:\ell)}, \mathcal{T}^{(p:\ell)})$
  - 8:  $(\hat{\mathbf{X}}^{(\ell+1)}, \dots, \hat{\mathbf{X}}^{(q)}) \leftarrow \text{BSMHF}(\mathbf{M}^{(\ell+1:q)}, \mathcal{T}^{(\ell+1:q)})$
- 

化后的蝶形稀疏矩阵分层分解矩阵。需要指出的是，若因子不需要满足约束 2，则该算法的结果退化为 C-T 基-2 分解算法。BSMHF 每次根据树索引产生两个中间矩阵，则对于  $2^t$  维度的  $\mathbf{F}_N$  需要  $t - 1$  次。

下一步进行对  $\beta$  的优化，对应的优化问题为当  $\mathbf{A}_k$  固定时的问题(6.49)。仍然考虑对  $\beta$  的梯度，我们有：

$$\nabla_\beta = \beta \text{tr}(\mathbf{F}) - \text{tr}\left(\prod_k \mathbf{A}_k\right)$$

由于  $\beta > 0$  且为实数，可采取的迭代策略为  $\beta \leftarrow \beta - \lambda |\nabla_\beta|$ 。事实上，由于  $\mathbf{F}_N$  的逐元素在单位圆内， $\beta$  的最佳取值位于  $(0, 1)$  之间，可以采用固定步长搜索的方式确定  $\beta$  的值。

还需要指出的是，由于蝶形因子也属于蝶形结构矩阵，可以实现对蝶形因子进行分层量化分解。以  $N = 2^3$  为例，算法可以将以 C-T 基-2 分解  $\mathbf{F}_8$  得到的  $\mathbf{A}_3, \mathbf{A}_2, \mathbf{A}_1$  中不满足约束 2 的  $\mathbf{A}_3$  矩阵再次拆分为三个新的蝶形结构矩阵  $\mathbf{A}_3^3, \mathbf{A}_3^2, \mathbf{A}_3^1$ （因为  $\mathbf{A}_3$  也是蝶形结构矩阵！）。然而，由于元素取值的离散性，元素将会在多次重复迭代后逐渐映射为  $\pm 1, \pm j, \pm 1 \pm j$ ，但是对于问题 5 中不限制  $q$  的上限时就不会出现模糊性。我们将在结果中展示，若使用上述算法对经过 C-T 基-2 分解得到的包含无理数的  $\mathbf{A}_k$  进行蝶形稀疏矩阵量化分解，其逼近误差将会好于直接对  $\mathbf{F}_N \mathbf{P}_N$  进行蝶形稀疏矩阵量化分解。

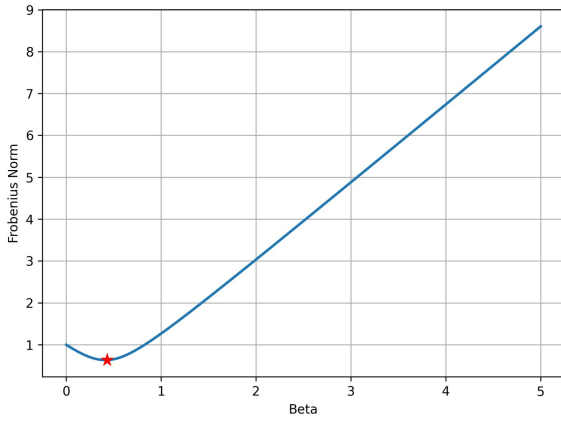
### 6.3.3 算法结果分析

对于  $N = 2$  的情况，DFT 矩阵不用分解，并且满足约束 1 和 2。我们分别对  $N = 2^t$ ,  $t = 2, 3, 4, 5$  的 DFT 矩阵  $\mathbf{F}_N$  使用蝶形稀疏矩阵分层量化分解算法，结果展示在下表中。

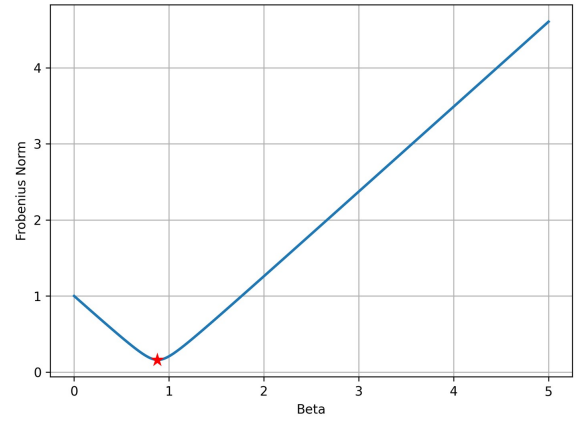
表 6.6 问题三结果

| 序号 | $\mathbf{F}_N$    | 分解个数 | 映射方法    | RMSE     | 硬件复杂度 | $\beta$ | 备注                                                                                                                                                                                                                                                                                                   |
|----|-------------------|------|---------|----------|-------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | $\mathbf{F}_2$    | 1    |         | 0        | 0     | 1       | 无需分解即满足约束                                                                                                                                                                                                                                                                                            |
| 2  | $\mathbf{F}_4$    | 3    | nearest | 0        | 0     | 1       | 将 $\mathbf{F}_4$ 分解为 $\mathbf{A}_2\mathbf{A}_1\mathbf{P}_4^\top$                                                                                                                                                                                                                                     |
| 3  | $\mathbf{F}_8$    | 4    | nearest | 0.37562  | 0     | 0.93    | 将 $\mathbf{F}_8$ 分解为 $\mathbf{A}_3\mathbf{A}_2\mathbf{A}_1\mathbf{P}_8^\top$                                                                                                                                                                                                                         |
| 4  | $\mathbf{F}_8$    | 6    | nearest | 0.16045  | 0     | 0.88    | 将 $\mathbf{F}_8$ 分解为 $\{\mathbf{A}_3^3\mathbf{A}_3^2\mathbf{A}_3^1\}\mathbf{A}_2\mathbf{A}_1\mathbf{P}_8^\top$                                                                                                                                                                                       |
| 5  | $\mathbf{F}_{16}$ | 5    | nearest | 0.6048   | 0     | 0.47    | 将 $\mathbf{F}_{16}$ 分解为 $\mathbf{A}_4\mathbf{A}_3\mathbf{A}_2\mathbf{A}_1\mathbf{P}_{16}^\top$                                                                                                                                                                                                       |
| 6  | $\mathbf{F}_{16}$ | 10   | nearest | 0.33     | 0     | 0.72    | 将 $\mathbf{F}_{16}$ 分解为 $\{\mathbf{A}_4^4\mathbf{A}_4^3\mathbf{A}_4^2\mathbf{A}_4^1\}\{\mathbf{A}_3^4\mathbf{A}_3^3\mathbf{A}_3^2\mathbf{A}_3^1\}\mathbf{A}_2\mathbf{A}_1\mathbf{P}_{16}^\top$                                                                                                       |
| 7  | $\mathbf{F}_{32}$ | 7    | nearest | 0.6335   | 0     | 0.41    | 将 $\mathbf{F}_{32}$ 分解为 $\mathbf{A}_5\mathbf{A}_4\mathbf{A}_3\mathbf{A}_2\mathbf{A}_1\mathbf{P}_{32}^\top$                                                                                                                                                                                           |
| 8  | $\mathbf{F}_{32}$ | 18   | nearest | 0.381822 | 0     | 0.688   | 将 $\mathbf{F}_{32}$ 分解为 $\{\mathbf{A}_5^5\mathbf{A}_5^4\mathbf{A}_5^3\mathbf{A}_5^2\mathbf{A}_5^1\}\{\mathbf{A}_4^5\mathbf{A}_4^4\mathbf{A}_4^3\mathbf{A}_4^2\mathbf{A}_4^1\}\{\mathbf{A}_3^5\mathbf{A}_3^4\mathbf{A}_3^3\mathbf{A}_3^2\mathbf{A}_3^1\}\mathbf{A}_2\mathbf{A}_1\mathbf{P}_{32}^\top$ |

如表6.6所示, 对于  $\mathbf{F}_8, \mathbf{F}_{16}, \mathbf{F}_{32}$ , 首先经过一轮不含投影的 BSMHF 分解后 (退化为 C-T 基-2 分解), 再将其中得到的包含无理数的因子进行投影的 BSMHF 分解后的精度均得到了显著提升。这暗示可以通过增加逼近矩阵数量的方法来获得 RMSE, 但可能引起硬件复杂度的上升。另一方面, 还可以执行混合方法, 即对部分因子实行投影, 而对部分中间矩阵不实行投影, 这在未来也是一个值得研究的问题。



(a) 序号3  $\beta = 0.93$



(b) 序号4  $\beta = 0.88$

图 6.8 迭代分解 RMSE 与  $\beta$  的关系

图6.8向我们展示了在 BSMHF 分解最后一步搜索  $\beta$  和 RMSE 的关系, 通过定步长搜索,  $\beta$  在预计的 (0, 1) 范围内存在最优值, 并当  $\beta > 1$  时将要扩大 RMSE 的结果。最后, 列出我们算法的分解结果如下所示, 碍于篇幅限制, 部分  $32 \times 32$  的矩阵不在其中, 以 mat 文件后缀形式存储。

1. 实验序号 1 分解矩阵：略。
2. 实验序号 2 分解矩阵：略。
3. 实验序号 3 分解矩阵： $\mathbf{F}_8 = \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1 \mathbf{P}_8^\top$ ，其中

$$\mathbf{P}_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (6.50)$$

$$\mathbf{A}_1 = \begin{bmatrix} (1+0j) & (1+0j) & 0j & 0j & 0j & 0j & 0j & 0j \\ (-1+0j) & (1+0j) & 0j & 0j & 0j & 0j & 0j & 0j \\ 0j & 0j & (-1+0j) & (-1+0j) & 0j & 0j & 0j & 0j \\ 0j & 0j & 1j & -1j & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (1+0j) & (1+0j) & 0j & 0j \\ 0j & 0j & 0j & 0j & (-2+0j) & (2+0j) & 0j & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & (-1+0j) & (-1+0j) \\ 0j & 0j & 0j & 0j & 0j & 0j & (1+1j) & (-1-1j) \end{bmatrix} \quad (6.51)$$

$$\mathbf{A}_2 = \begin{bmatrix} (-1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j \\ (-1+0j) & 0j & (-1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & (-1+0j) & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (-1+0j) & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & 0j & (1+0j) \\ 0j & 0j & 0j & 0j & 1j & 0j & 1j & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & 0j & -1j \end{bmatrix} \quad (6.52)$$

$$\mathbf{A}_3 = \begin{bmatrix} (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j \\ 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j & 0j \\ 0j & 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j \\ 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) \\ (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j \\ 0j & 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) \end{bmatrix} \quad (6.53)$$

4. 实验序号 4 分解矩阵:  $\mathbf{F}_8 = \{\mathbf{A}_3^3 \mathbf{A}_3^2 \mathbf{A}_3^1\} \mathbf{A}_2 \mathbf{A}_1 \mathbf{P}_8^\top$ , 其中

$$\mathbf{A}_3^3 = \begin{bmatrix} (-1+0j) & 0j & 0j & 0j & 0j & 0j & 0j & 0j \\ 0j & (-1+0j) & 0j & 0j & 0j & 0j & 0j & 0j \\ 0j & 0j & (-1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & 0j & (-1+0j) & 0j & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & (-1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & 0j & (1+0j) \end{bmatrix} \quad (6.54)$$

$$\mathbf{A}_3^2 = \begin{bmatrix} (1+0j) & 0j & 0j & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j & 0j \\ 0j & 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & 0j & (-1+1j) & 0j & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & 1j & 0j \\ 0j & 0j & 0j & 0j & 0j & 0j & 0j & (-1-1j) \end{bmatrix} \quad (6.55)$$

$$\mathbf{A}_3^1 = \begin{bmatrix} (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j & 0j \\ 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j & 0j \\ 0j & 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j & (1+0j) \\ (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j \\ 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j & 0j \\ 0j & 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) & 0j \\ 0j & 0j & 0j & (-1+0j) & 0j & 0j & 0j & (-1+0j) \end{bmatrix} \quad (6.56)$$

$$\mathbf{A}_2 = \begin{bmatrix} (1+0j) & 0j & (1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & -1j & 0j & 0j & 0j & 0j \\ (1+0j) & 0j & (-1+0j) & 0j & 0j & 0j & 0j & 0j \\ 0j & (1+0j) & 0j & +1j & 0j & 0j & 0j & 0j \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & (1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & (1+0j) & 0j & -1j \\ 0j & 0j & 0j & 0j & (1+0j) & 0j & (-1+0j) & 0j \\ 0j & 0j & 0j & 0j & 0j & (1+0j) & 0j & +1j \end{bmatrix} \quad (6.57)$$

$$\mathbf{A}_1 = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix} \quad (6.58)$$

$\mathbf{P}_8$  同实验序号 3。

5. 实验序号 5-8: 限于篇幅原因, 不再展示。



## 7 问题 4 分析与求解

### 7.1 问题 4 分析

由于

$$\mathbf{F}_4 = \mathbf{A}_{2,2}\mathbf{A}_{2,1}\mathbf{P}_4^\top, \quad \mathbf{F}_8 = \mathbf{A}_{3,3}\mathbf{A}_{3,2}\mathbf{A}_{3,1}\mathbf{P}_8^\top \quad (7.59)$$

都已经在问题 3 中得到了逼近，问题 4 的任务相对简单。我们有

$$\begin{aligned} \mathbf{F}_4 \otimes \mathbf{F}_8 &= (\mathbf{A}_{2,2}\mathbf{A}_{2,1}\mathbf{P}_4^\top) \otimes (\mathbf{A}_{3,3}\mathbf{A}_{3,2}\mathbf{A}_{3,1}\mathbf{P}_8^\top) \\ &= (\mathbf{I}_4\mathbf{A}_{2,2}\mathbf{I}_4\mathbf{A}_{2,1}\mathbf{I}_4\mathbf{P}_4^\top) \otimes (\mathbf{A}_{3,3}\mathbf{I}_8\mathbf{A}_{3,2}\mathbf{I}_8\mathbf{A}_{3,1}\mathbf{P}_8^\top) \\ &= (\mathbf{I}_4 \otimes \mathbf{A}_{3,3}) (\mathbf{A}_{2,2} \otimes \mathbf{I}_8) (\mathbf{I}_4 \otimes \mathbf{A}_{3,2}) (\mathbf{A}_{2,1} \otimes \mathbf{I}_8) (\mathbf{I}_4 \otimes \mathbf{A}_{3,1}) (\mathbf{P}_4^\top \otimes \mathbf{P}_8^\top) \end{aligned} \quad (7.60)$$

注意到式 (7.60) 中只有  $\mathbf{A}_{3,3}$  需要近似，其余矩阵都是精确的，并且满足约束 1。

实际上，任意矩阵  $\mathbf{F}_{N_1} \otimes \mathbf{F}_{N_2}$  ( $N_1, N_2 = 2^{t_1}, 2^{t_2}$ ) 都可以在定理 1 的 C-T 基-2 分解的基础上利用 Kronecker 积的性质分解成  $\mathbf{I}_{N_1} \otimes \mathbf{A}_n$  或者  $\mathbf{A}_m \otimes \mathbf{I}_{N_2}$  的乘积。

**定理 2** 对于任意满足  $N_1, N_2 = 2^{t_1}, 2^{t_2}$  的 DFT 矩阵

$$\mathbf{F}_{N_1} = \mathbf{A}_{t_1,t_1}\mathbf{A}_{t_1,t_1-1}\mathbf{A}_{t_1,t_1-2}\cdots\mathbf{A}_{t_1,1}\mathbf{P}_{N_1}^\top \quad (7.61)$$

$$\mathbf{F}_{N_2} = \mathbf{A}_{t_2,t_2}\mathbf{A}_{t_2,t_2-1}\mathbf{A}_{t_2,t_2-2}\cdots\mathbf{A}_{t_2,1}\mathbf{P}_{N_2}^\top \quad (7.62)$$

$\mathbf{F}_{N_1} \otimes \mathbf{F}_{N_2}$  都可以写成

$$\begin{aligned} &(\mathbf{A}_{t_1,t_1} \otimes \mathbf{I}_{N_2}) (\mathbf{A}_{t_1,t_1-1} \otimes \mathbf{I}_{N_2}) \cdots (\mathbf{A}_{t_1,1} \otimes \mathbf{I}_{N_2}) (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2,t_2}) \\ &\times (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2,t_2-1}) \cdots (\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2,1}) (\mathbf{P}_{N_1}^\top \otimes \mathbf{P}_{N_2}^\top) \end{aligned} \quad (7.63)$$

的形式，并且任意  $\mathbf{I}_{N_1} \otimes \mathbf{A}_{t_2,n}$  或者  $\mathbf{A}_{t_1,m} \otimes \mathbf{I}_{N_2}$  都满足约束 1。

#### 证明 1

$$\mathbf{F}_{N_1} = \mathbf{A}_{t_1,t_1}\mathbf{A}_{t_1,t_1-1}\mathbf{A}_{t_1,t_1-2}\cdots\mathbf{A}_{t_1,1}\mathbf{I}_{N_1}^{t_2}\mathbf{P}_{N_1}^\top \quad (7.64)$$

$$\mathbf{F}_{N_2} = \mathbf{I}_{N_2}^{t_1}\mathbf{A}_{t_2,t_2}\mathbf{A}_{t_2,t_2-1}\mathbf{A}_{t_2,t_2-2}\cdots\mathbf{A}_{t_2,1}\mathbf{P}_{N_2}^\top \quad (7.65)$$

根据定理 1，C-T 基-2 分解得到的  $\mathbf{A}_k$  一定满足稀疏矩阵约束 1，所以  $\mathbf{A}_k \otimes \mathbf{I}_N$  和  $\mathbf{I}_N \otimes \mathbf{A}_k$  也一定满足约束 1。Q.E.D.

### 7.2 丢番图逼近

对于  $\mathbf{A}_{3,3}$  的第二个对角矩阵 (6.38c)，我们的任务是使用满足约束 2 的  $2 \times 2$  实对角矩阵 (不改变方向只改变长度) 的乘积来逼近  $\text{diag}(\sqrt{2}, 1)$ ，为此我们需要至少两个不同的具有素数元素的对角矩阵。

**定理 3** 令  $G$  表示乘法群  $(\mathbb{R}_+, \times)$ ，其元素为正实数的全体，元素之间采用标准欧氏距离。令  $P$  表示  $G$  的一个子群，如果  $P$  中至少包含两个素数，那么  $P$  在  $G$  中稠密。

**证明 2 (定理 3)** 设  $p, q$  是  $P$  中两个相异的素数，那么根据 *Gelfond–Schneider* 定理， $\frac{\ln p}{\ln q}$  是超越数，因此是无理数。所以根据 *Dirichlet* 逼近定理， $\ln p, \ln q$  生成的加法子群在  $(\mathbb{R}, +)$  中稠密，因此  $p = e^{\ln p}, q = e^{\ln q}$  生成的乘法子群在  $(\mathbb{R}_+, \times)$  中稠密，任意  $x \in \mathbb{R}$  可以被  $\{p^n q^m | m, n \in \mathbb{Z}\}$  以任意精度逼近。Q.E.D.

满足约束 2 的包含素因子的对角矩阵共有 5 种，分别包含 2, 3, 5, 7, 17。其中  $\mathbf{D}_{2,1} = \text{diag}(2, 1)$  与自身相乘 1 次的乘法复杂度为  $L(\mathbf{D}_{2,1}\mathbf{D}_{2,1}) = 1$ ，其余 4 个矩阵的生成方式和与自身相乘 1 次的复杂度如下

$$\mathbf{D}_{3,1} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad L(\mathbf{D}_{3,1}\mathbf{D}_{3,1}) = 1 \quad (7.66)$$

$$\mathbf{D}_{5,1} = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2+j & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2-j & 0 \\ 0 & 1 \end{bmatrix}, \quad L(\mathbf{D}_{5,1}\mathbf{D}_{5,1}) = 3 \quad (7.67)$$

$$\mathbf{D}_{7,1} = \begin{bmatrix} 7 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 4 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 4 & -1 \end{bmatrix}, \quad L(\mathbf{D}_{7,1}\mathbf{D}_{7,1}) = 4 \quad (7.68)$$

$$\mathbf{D}_{17,1} = \begin{bmatrix} 17 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 4+j & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4-j & 0 \\ 0 & 1 \end{bmatrix}, \quad L(\mathbf{D}_{17,1}\mathbf{D}_{17,1}) = 3 \quad (7.69)$$

例如，我们只使用 2 和 3 来逼近  $\sqrt{2}$  (可以达到任意精度):

$$3^a = 2^b \sqrt{2} \implies a \ln 3 = (b + \frac{1}{2}) \ln 2 \implies \frac{2a}{2b+1} = \frac{\ln 2}{\ln 3} \quad (7.70)$$

而  $\frac{\ln 2}{\ln 3} \approx 0.6309$  的连分数收敛列为

$$0, 1, \frac{1}{2}, \frac{2}{3}, \frac{5}{8}, \frac{12}{19}, \frac{41}{65}, \frac{53}{84}, \frac{306}{485}, \frac{665}{1054}, \frac{15601}{24727}, \dots \quad (7.71)$$

$$\frac{31867}{50508}, \frac{79335}{125743}, \frac{111202}{176251}, \frac{190537}{301994}, \frac{10590737}{16785921}, \frac{10781274}{17087915}, \dots$$

注意到  $2a$  是偶数而  $2b+1$  是奇数，可选的分数有

$$\frac{2}{3} \approx 0.6667, \frac{12}{19} \approx 0.6316, \frac{306}{485} \approx 0.6309, \frac{111202}{176251} \approx 0.6309, \frac{10781274}{17087915} \approx 0.6309 \quad (7.72)$$

对应于  $\frac{3^a}{2^b}$  为 ( $\sqrt{2} \approx 1.4142$ )

$$\frac{3}{2} = 1.5, \frac{3^6}{2^9} \approx 1.4238, \frac{3^{153}}{2^{242}} \approx 1.4135, \frac{3^{55601}}{2^{88125}} \approx 1.4142, \frac{3^{5390637}}{2^{8543957}} \approx 1.4142 \quad (7.73)$$

其中  $\frac{36}{29}$  的逼近效果已经较好, 其误差约为 0.0096, 乘法复杂度约为  $L(\mathbf{D}_{3,1}^6 \mathbf{D}_{2,1}^9) = 6+9 = 15$ 。还可以通过将两个  $\mathbf{D}_{2,1}$  合并为一个  $\mathbf{D}_{4,1}$  来进一步降低复杂度, 此处略去。

接下来使用 2, 3, 5, 7, 17 来逼近, 得到  $2^a 3^b 5^c 7^d 17^e = \sqrt{2}$ , 其乘法复杂度接近于  $L = |a| + |b| + 3|c| + 4|d| + 3|e|$ 。因此我们可以提出一个整数规划问题, 最小化的目标函数是

$$Q = \frac{1}{2} \left( 2^a 3^b 5^c 7^d 17^e - \sqrt{2} \right)^2 \quad (7.74)$$

其中

$$2^a 3^b 5^c 7^d 17^e \in \bigcup_{n=1}^{\infty} \{2, 3, 5, 7, 17, \frac{1}{2}, \frac{1}{3}, \frac{1}{5}, \frac{1}{7}, \frac{1}{17}\}^n, \quad a, b, c, d, e \in \mathbb{Z} \quad (7.75)$$

约束条件为乘法复杂度不大于某个上限:  $L = |a| + |b| + 3|c| + 4|d| + 3|e| \leq L_{max}$

### 7.3 丢番图逼近的整数规划问题求解

采用遗传算法解整数规划问题 (7.74), 得到精度极高的三组解

1.  $a = -12, b = 0, c = -1, d = -2, e = 5$ , 误差  $|2^a 3^b 5^c 7^d 17^e - \sqrt{2}| \approx 6.6307 \times 10^{-4}$
2.  $a = 16490, b = -6366, c = -6595, d = 8188, e = -3443$ , 误差  $\approx 4.6473 \times 10^{-4}$
3.  $a = -632, b = -12912, c = 3900, d = 1162, e = 2148$ , 误差  $\approx 3.5606 \times 10^{-4}$

第 2, 3 组解没有实际意义, 这里仅就第 1 组解构造  $\mathbf{A}_{3,3}$  的逼近。由于  $4^{-6} 5^{-1} 7^{-2} 17^5 \approx \sqrt{2}$ , 我们有

$$\text{diag}(\sqrt{2}, 1) \approx \frac{1}{4^6 5^1 7^2} \mathbf{D}_{17,1}^5 \mathbf{D}_{1,4}^6 \mathbf{D}_{1,5} \mathbf{D}_{1,7}^2 \quad (7.76)$$

因此式 (6.38c) 中的第二个对角矩阵逼近式为

$$\begin{aligned} & \text{diag}(\sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}, 1, \sqrt{2}, 1) \quad (\beta_{8d} = \frac{1}{4^6 5^1 7^2}, \text{但是注意最后还要除以 } \sqrt{2}) \\ & \approx \beta_{8d} [\text{diag}(17, 17, 17, 17, 17, 1, 17, 1)]^5 \times [\text{diag}(1, 1, 1, 1, 1, 4, 1, 4)]^6 \\ & \times \text{diag}(1, 1, 1, 1, 1, 5, 1, 5) \times [\text{blkdiag}(\mathbf{I}_4, \mathbf{D}_{1,7}, \mathbf{D}_{1,7})]^2 \end{aligned} \quad (7.77)$$

其中各素因子矩阵的满足约束 2 ( $q = 3$ ) 的生成方法见 (7.2), 使用该逼近式得到的  $\text{RMSE}(\hat{\mathbf{A}}_{3,3}) \approx 2.0302 \times 10^{-4}$ 。

### 7.4 问题 4 的高精度求解

如果采用式 (6.44) 的遍历性定理逼近结果

$$\mathbf{F}_4 \otimes \hat{\mathbf{F}}_8 = \left( \mathbf{I}_4 \otimes \hat{\mathbf{A}}_{3,3} \right) (\mathbf{A}_{2,2} \otimes \mathbf{I}_8) (\mathbf{I}_4 \otimes \mathbf{A}_{3,2}) (\mathbf{A}_{2,1} \otimes \mathbf{I}_8) (\mathbf{I}_4 \otimes \mathbf{A}_{3,1}) (\mathbf{P}_4^\top \otimes \mathbf{P}_8^\top) \quad (7.78)$$

其具体数值见附录 B,

$$\text{RMSE}(\mathbf{F}_4 \otimes \hat{\mathbf{F}}_8) = \frac{1}{32} \|\mathbf{F}_4 \otimes \mathbf{F}_8 - \mathbf{F}_4 \otimes \hat{\mathbf{F}}_8\|_F \approx 0.0682, \quad \beta_{4,8} = \frac{1}{5\sqrt{85}} \quad (7.79)$$

复杂度结果为  $L = 1024, q = 3, C = q \times L = 3072$ 。其 RMSE 和  $\beta$  与式 (6.46) 相同，这并不意外，因为两者使用了同样的  $\mathbf{A}_{3,3}$  近似，而其余矩阵都是精确的。

如果采用式 (7.77) 的丢番图逼近结果， $\text{RMSE}(\mathbf{F}_4 \otimes \hat{\mathbf{F}}_8) \approx 4.0605 \times 10^{-4}$ ，但是其乘法复杂度太高，不具有实际意义，仅为证明任意精度逼近的可行性。

## 8 问题 5 分析与求解

### 8.1 问题 5 分析

本题中不再限制  $q$  的范围, 正如问题 3 中所提到的那样, 由于因子  $\beta$  的出现, 可以通过设计很大的数  $M$ , 使得  $\beta = 2^M$ , 那么可以允许元素的取值来到  $x + yj, \forall x, y \in \{0, 2^{1-M}, \dots, 2^{q-M}\}$ 。注意到 2 的幂次为整数倍, 可以允许其具有更丰富的选择范围。通过改进问题 3 中的固定支持矩阵量化分解算法, 我们提出基于[分布式遗传算法的整数规划改进策略](#)。此外, 我们还继续拓展了解析法的适应范围, 使其能够实现对任意 DFT 矩阵以任意精度逼近。我们首先介绍解析方法, 然后介绍改进的优化算法。

### 8.2 任意 DFT 矩阵的任意精度逼近

在遍历性定理[6.2.1](#)和丢番图逼近[3](#)的基础上, 我们终于可以以任意精度对任意 DFT 矩阵进行逼近。首先我们证明

**引理 3**  $\arctan 2$  和  $\frac{\pi}{4} = \arctan 1$  在有理数域  $\mathbb{Q}$  上是线性无关的。

**证明 3** 因为  $\frac{\pi}{4} = \arctan 1$  是  $\pi$  的有理数倍, 问题转化为令  $\beta = \arctan 2$ , 是否存在一个非零整数  $m$  使得

$$m\beta \equiv 0 \pmod{\pi}$$

如果确实存在  $m$ , 我们有

$$1 = (e^{2j\beta})^m = (\cos 2\beta + j \sin 2\beta)^m = \left(-\frac{3}{5} + j\frac{4}{5}\right)^m = \left(\frac{-3+4j}{5}\right)^m \quad (8.80)$$

但是因为  $5, -3+4j$  都是  $\mathbb{Z}[j]$  中的不可约元, 而  $\mathbb{Z}[j]$  是唯一分解整环, 这是不可能的。Q.E.D.

引理[3](#)表明旋转角度组合  $(\frac{\pi}{4}, \arctan 2)$  是遍历的, 因此我们可以以任意精度获得任意角度差, 但是  $\arctan 2$  和  $\frac{\pi}{4}$  对应的满足约束 2 ( $q = 3$ ) 的元素具有不同的长度。不过这不是问题, 因为丢番图逼近[3](#)使我们能够以任意精度调整两个元素的长度, 因此

**引理 4** 对于任意  $z \in \mathbb{C}$  以及任意  $\varepsilon > 0$ , 存在实数  $\beta$  以及有限个满足约束 2 ( $q = 3$ ) 的  $2 \times 2$  对角矩阵  $\mathbf{D}_1, \mathbf{D}_2, \dots, \mathbf{D}_n$  满足

$$\beta \mathbf{D}_1 \mathbf{D}_2 \dots \mathbf{D}_n = \begin{bmatrix} 1 & 0 \\ 0 & y \end{bmatrix} \quad (8.81)$$

并且  $|y - z| < \varepsilon$ 。特别地, 任何单位圆上的元素  $w \in S^1$  可以被任意逼近。

**定理 4** 对于任意 DFT 矩阵  $\mathbf{F}_N$  以及任意  $\varepsilon > 0$ , 存在实数  $\beta$  以及有限个满足约束 2 ( $q = 3$ ) 的矩阵  $\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k$ , 使得

$$\text{RMSE}(\beta \mathbf{A}_1 \mathbf{A}_2 \dots \mathbf{A}_k) = \frac{1}{N} \|\mathbf{F}_N - \beta \mathbf{A}_1 \mathbf{A}_2 \dots \mathbf{A}_k\|_F < \varepsilon \quad (8.82)$$

**证明 4 (概述)** 根据  $C$ - $T$  基-2 分解定理4.9,

$$\mathbf{F}_n = \mathbf{A}_t \dots \mathbf{A}_1 \mathbf{P}_n^\top \quad (8.83)$$

$$\mathbf{A}_t = \mathbf{I}_r \otimes \mathbf{B}_N = \mathbf{I}_r \otimes (\mathbf{P}_\Lambda \Lambda) \quad (8.84)$$

其中  $\Lambda$  为对角阵, 因此可以被引理4以任意精度逼近, 而  $\mathbf{I}_r$  和  $\mathbf{P}_\Lambda$  都是精确的, 并且因为任意有限维矩阵的 *Frobenius* 范数都是有界的, 只要使用的矩阵足够多, 最终一定有  $\text{RMSE}(\beta \mathbf{A}_1 \mathbf{A}_2 \dots \mathbf{A}_k)$  小于任意  $\varepsilon > 0$ . *Q.E.D.*

作为定理4的应用, 我们计算  $\mathbf{F}_{16}$  的高精度逼近

$$\mathbf{F}_{16} = \mathbf{A}_{4,4} \mathbf{A}_{4,3} \mathbf{A}_{4,2} \mathbf{A}_{4,1} \mathbf{P}_{16}^\top \quad (8.85)$$

其中

$$\mathbf{A}_{4,1} = \mathbf{I}_8 \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \mathbf{A}_{4,2} = \mathbf{I}_4 \otimes \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -j \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & j \end{bmatrix}, \mathbf{A}_{4,3} = \mathbf{I}_2 \otimes \mathbf{A}_{3,3} \quad (8.86)$$

全部已知 ( $\mathbf{A}_{3,3}$  采用7.77的高精度逼近), 而

$$\mathbf{A}_{4,4} = \begin{bmatrix} \mathbf{I}_8 & \Omega_8 \\ \mathbf{I}_8 & -\Omega_8 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_8 & \mathbf{I}_8 \\ \mathbf{I}_8 & -\mathbf{I}_8 \end{bmatrix} \begin{bmatrix} \mathbf{I}_8 & \mathbf{0} \\ \mathbf{0} & \Omega_8 \end{bmatrix} \quad (8.87)$$

$$\Omega_8 = \text{diag}(1, w_{16}, w_{16}^2, w_{16}^3, w_{16}^4, w_{16}^5, w_{16}^6, w_{16}^7), \quad w_{16} = e^{-j2\pi/16} = \frac{\sqrt{2+\sqrt{2}}}{2} - j \frac{\sqrt{2-\sqrt{2}}}{2} \quad (8.88)$$

$$\begin{aligned} \text{diag}(1, w_{16}) &\approx \text{diag}(1, 0.9239 - 0.3827j) \approx \frac{1}{4^6 5^1 \cdot 7^2 17^4 \cdot \sqrt{2}} \mathbf{D}_{1,-1-j} \mathbf{D}_{1,4+j}^8 \mathbf{D}_{17,1}^9 \mathbf{D}_{1,4}^6 \mathbf{D}_{1,5} \mathbf{D}_{1,7}^2 \\ &\approx \text{diag}(1.0005, 0.9225 - 0.3861j) \end{aligned} \quad (8.89)$$

由于  $\text{diag}(1, w_{16}^2) \approx [\text{diag}(1.0005, 0.9225 - 0.3861j)]^2$ , 其余  $w_{16}^3, w_{16}^4 = -j, \dots, w_{16}^7$  同理也都是满足约束 2 的精确值或者已经通过  $\text{diag}(1, w_{16})$  得到逼近, 因此  $\Omega_8$  可以完全由满足约束 2 的矩阵表示出来。最终

$$\text{RMSE}(\hat{\mathbf{F}}_{16}) = \frac{1}{16} \|\mathbf{F}_{16} - \hat{\mathbf{F}}_{16}\|_F = 0.0109 \quad (8.90)$$

最终逼近的  $\mathbf{F}_{16}$  矩阵的实部和虚部为:



### 8.3 分布式遗传算法的整数规划改进策略

在本节中，我们提出的一种大范围搜索策略是将矩阵的每一个元素设置为  $\pm 2^p \pm 2^q j$ ,  $p, q \in \mathbb{Z}$ ，其中  $p, q$  为待规划的未知整数，目标仍然是使得 Frobenius 范数最小。对于一个  $N$  阶  $\mathbf{F}_N$  矩阵，共有  $2N^2$  个未知变量。若考虑符号，则未知变量的个数将增加到  $4N^2$ ，这样直接求解无疑是困难的。

基于问题 3 中的算法 5 和蝶形因子的优势，我们可以对固定支持矩阵量化分解方法进行改进。其核心思想是利用  $\mathbf{S}^i$  函数的切片性质，将目标矩阵  $\mathbf{Z}$  分解为  $\mathbf{Z} = \sum_i \mathbf{Z} \odot \mathbf{S}^i$ 。在每一步，通过对幂次的整数规划，设计  $\mathbf{x}_i \mathbf{y}_i^\top$  使得满足  $\arg\min\{\frac{1}{2}\|\mathbf{Z} \odot \mathbf{S}^i - \mathbf{x}_i \mathbf{y}_i^\top\|_F^2\}$ 。注意，我们可以通过  $\beta$  放缩来使得  $p \in \mathbb{Z}$ 。得益于  $\mathbf{S}$  因子的特性，若要使得  $\mathbf{X}$  中的元素恢复到  $2^p, p \in \mathbb{N}^+$  中，只需要设计  $\beta = 2^r, r = -\log(\min\{\mathbf{X}\})$ 。设  $\mathbf{x}_i, \mathbf{y}_i$  中待设计元素的个数分别为  $N_x$  和  $N_y$ ，考虑到元素可能为复数，因此需要设计  $2 \times (N_x + N_y)$  个整数决策变量，记决策变量为  $\mathbf{z} \in \mathbb{Z}^{2 \times (N_x + N_y)}$ 。因为对  $\mathbf{Z} \odot \mathbf{S}^i$  的 SVD 分解的最大贡献很好地继承了符号特性，可以将符号默认地等同于 SVD 分解结果的  $\mathbf{u}_1$  和  $\mathbf{v}_1$ 。或者，也可以为每个元素增设两个 0-1 决策变量以选择符号。上述过程的伪代码形式见算法 7。

---

#### 算法 7 改进的固定支持矩阵量化分解

---

**输出:** FSMQF( $\mathbf{Z} \in \mathbb{C}^{m \times n}, \mathbf{S}^L \in \mathbb{B}^{m \times r}, \mathbf{S}^R \in \mathbb{B}^{n \times r}$ )

---

- 1:  $(\mathbf{S}^i)_{i=1}^N \leftarrow \varphi(\mathbf{S}^L, \mathbf{S}^R)$
  - 2: **for**  $i = 1 \in \llbracket r \rrbracket$  **do**
  - 3:      $(\mathbf{u}, \sigma, \mathbf{v}) = \text{SVD}(\mathbf{Z} \odot \mathbf{S}^i)$
  - 4:      $N_L \leftarrow \sum \mathbf{S}^L(:, i), N_R \leftarrow \sum \mathbf{S}^R(:, i)$
  - 5:     设置初始种群数量为  $N = 2(N_L + N_R)$  # 开始遗传算法
  - 6:      $(\mathbf{x}_i, \mathbf{y}_i) \leftarrow \arg\min\{\frac{1}{2}\|\mathbf{Z} \odot \mathbf{S}^i - \mathbf{x}_i \mathbf{y}_i^\top\|_F^2\}$ , 满足  $\text{supp}(\mathbf{x}_i \mathbf{y}_i^\top) \subseteq \mathbf{S}^i$ , 其中  $\mathbf{x}_i(k) = \text{sgn}(\mathbf{u}_i)(2^{p_x} + 2^{q_x} j), \mathbf{y}_i(k) = \text{sgn}(\mathbf{v}_i)(2^{p_y} + 2^{q_y} j)$ ,  $p_x, p_y, q_x, q_y$  是待规划的整数。
  - 7:  $\mathbf{X} \leftarrow (\mathbf{x}_1 \dots, \mathbf{x}_r) \in \mathbb{C}^{m \times r}$
  - 8:  $\mathbf{Y} \leftarrow (\mathbf{y}_1 \dots, \mathbf{y}_r) \in \mathbb{C}^{m \times r}$
- 

这样，通过在每一次循环中求解一个小规模变量的整数规划问题，求解速度将会非常快。最后，再应用定步长搜索的方法求解  $\beta$ 。不难发现，若设置  $p, q$  的最小值为 0，最大值为 4，则该方法退化为满足约束 2 下的临近投影法。

在求解时，可以通过对整数的上下界进行约束  $L_q \leq q \leq U_q, L_p \leq p \leq U_p$ ，则通过设置  $\beta = 2^{L_q}$ ，总可以使得  $2^{-L_q} \mathbf{X}$  的元素满足约束条件。

以  $\mathbf{F}_8$  矩阵为例，对  $\mathbf{F}_8$  矩阵中无法表达的 C-T 基-2 分解因子  $\mathbf{A}_3$  的整数规划蝴蝶因子分解，并设计  $p, q$  的上下界为  $\pm 10$ ，得到  $\mathbf{A}_3$  的估计结果如图 8.9 所示（稀疏矩阵表达）。对应的  $\beta$  最优值为 0.96，多次遗传算法的收敛值约为  $\text{RMSE} = 0.1337$ ，相比于问题 3 的结果



有所提升。硬件复杂度  $C \leq 108 \times 10 = 1728$ 。

以  $\mathbf{F}_{16}$  矩阵为例，对  $\mathbf{F}_{16}$  矩阵中无法表达的 C-T 基-2 分解因子  $\mathbf{A}_4$ ,  $\mathbf{A}_3$  的整数规划蝴蝶因子分解，并设计  $p, q$  的上下界为  $\pm 10$ ，得到  $\mathbf{A}_4$ ,  $\mathbf{A}_3$  的估计结果如图 8.10 所示（稀疏矩阵表达）。对应的  $\beta$  最优值为 1.01，多次遗传算法的收敛值约为  $\text{RMSE} = 0.1737$ ，相比于问题 3 的结果 0.3 有很高提升。硬件复杂度  $C \leq 776 \times 10 = 7760$ 。

上述算法的 MATLAB 代码见附录 3.4。

| $\mathbf{A}_3$          |           | $\mathbf{A}_3^3$         |          | $\mathbf{A}_3^2$        |          | $\mathbf{A}_3^1$        |
|-------------------------|-----------|--------------------------|----------|-------------------------|----------|-------------------------|
| (1,1) 1.0000 + 0.0000i  |           | (1,1) -0.0039 + 0.0000i  |          | 1.0e+03 *               |          | 1.0e+02 *               |
| (5,1) 1.0000 + 0.0000i  |           | (5,1) -0.0039 + 0.0000i  |          | (1,1) 0.0010 + 0.0000i  |          | (1,1) -2.5600 + 0.0000i |
| (2,2) 1.0000 + 0.0000i  |           | (2,2) -16.0000 + 0.0000i |          | (2,2) 0.0160 + 0.0000i  |          | (2,2) -0.0000 + 0.0000i |
| (6,2) 1.0000 + 0.0000i  |           | (6,2) -16.0000 + 0.0000i |          | (3,3) 0.0000 + 0.0000i  |          | (3,3) -0.0400 + 0.0000i |
| (3,3) 1.0000 + 0.0000i  |           | (3,3) -64.0000 + 0.0000i |          | (4,4) 0.0001 + 0.0000i  |          | (4,4) -0.3200 + 0.0000i |
| (7,3) 1.0000 + 0.0000i  | $\approx$ | (7,3) -64.0000 + 0.0000i | $\times$ | (5,5) 0.0040 + 0.0000i  | $\times$ | (5,5) 0.0800 + 0.0000i  |
| (4,4) 1.0000 + 0.0000i  |           | (4,4) -0.2500 + 0.0000i  |          | (6,6) -0.0000 - 0.0640i |          | (6,6) 0.0000 + 0.0000i  |
| (8,4) 1.0000 + 0.0000i  |           | (8,4) -0.2500 + 0.0000i  |          | (8,6) -0.0000 - 0.0000i |          | (7,7) -0.0100 + 0.0000i |
| (1,5) 1.0000 + 0.0000i  |           | (1,5) 0.0312 + 0.0000i   |          | (7,7) -0.0000 + 1.0240i |          | (8,7) -0.0000 + 0.0000i |
| (5,5) -1.0000 + 0.0000i |           | (5,5) -0.0312 + 0.0000i  |          | (8,8) 0.0005 - 0.0010i  |          | (8,8) 0.0000 + 2.5600i  |
| (2,6) 0.7071 - 0.7071i  |           | (2,6) 0.0156 + 0.0000i   |          |                         |          |                         |
| (6,6) -0.7071 + 0.7071i |           | (6,6) -0.0156 + 0.0039i  |          |                         |          |                         |
| (3,7) 0.0000 - 1.0000i  |           | (3,7) 0.0010 + 0.0000i   |          |                         |          |                         |
| (7,7) -0.0000 + 1.0000i |           | (7,7) -0.0010 + 0.0000i  |          |                         |          |                         |
| (4,8) -0.7071 - 0.7071i |           | (4,8) -0.0039 + 0.0000i  |          |                         |          |                         |
| (8,8) 0.7071 + 0.7071i  |           | (8,8) 0.0039 + 0.0010i   |          |                         |          |                         |

图 8.9  $\mathbf{F}_8$  的 C-T 基-2 分解矩阵  $\mathbf{A}_3$  的量化逼近

## 9 模型评价

本文针对  $N$  阶离散傅里叶矩阵的整数稀疏矩阵逼近问题，分别从解析和优化的视角给出了高精度和低运算复杂度的分解方法。两种方法都是基于蝶形因子分解（C-T 基-2 分解）的一般性结果的后端优化。两种方法的优点和缺点总结如下：

### 9.1 模型优点

- 解析方法：我们建立的解析方法可以实现对任意 DFT 矩阵的任意精度逼近，这对于部分元素受限同时追求高精度的场合至关重要。
- 优化方法：我们建立的模型和设计的算法最终能够实现对 DFT 类矩阵的整数逼近分解（见问题 5），并且具有潜在的扩展能力以面对任意类型的矩阵。算法具有高度可探索性，能够重复性地展开一类蝶形结构矩阵，直到达到逼近模糊。在对  $\mathbf{F}_8$  和  $\mathbf{F}_{16}$  的整数逼近上能够达到  $\text{RMSE}=0.1337$  和  $0.1737$  的精度。

### 9.2 模型缺点

- 解析方法：解析方法的高精度是建立在大量硬件复杂度基础上的。事实上，可以根据硬件复杂度的需求来实现精度和硬件复杂度之间的权衡。
- 优化方法：蝶形稀疏矩阵分解算法依赖于固定支持的蝶形因子结构矩阵，导致可能存

| $A_4$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |           | $A_4^4$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |          | $A_4^3$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |          | $A_4^2$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |          | $A_4^1$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (1,1) 1.0000 + 0.0000i<br>(9,1) 1.0000 + 0.0000i<br>(2,2) 1.0000 + 0.0000i<br>(10,2) 1.0000 + 0.0000i<br>(3,3) 1.0000 + 0.0000i<br>(11,3) 1.0000 + 0.0000i<br>(4,4) 1.0000 + 0.0000i<br>(12,4) 1.0000 + 0.0000i<br>(5,5) 1.0000 + 0.0000i<br>(13,5) 1.0000 + 0.0000i<br>(6,6) 1.0000 + 0.0000i<br>(14,6) 1.0000 + 0.0000i<br>(7,7) 1.0000 + 0.0000i<br>(15,7) 1.0000 + 0.0000i<br>(8,8) 1.0000 + 0.0000i<br>(16,8) 1.0000 + 0.0000i<br>(1,9) 1.0000 + 0.0000i<br>(9,9) -1.0000 + 0.0000i<br>(2,10) 0.9239 + 0.3827i<br>(10,10) -0.9239 + 0.3827i<br>(3,11) 0.7071 - 0.7071i<br>(11,11) -0.7071 + 0.7071i<br>(4,12) 0.3827 - 0.9239i<br>(12,12) -0.3827 + 0.9239i<br>(5,13) -0.0000 - 1.0000i<br>(13,13) 0.0000 + 1.0000i<br>(6,14) -0.3827 - 0.9239i<br>(14,14) 0.3827 + 0.9239i<br>(7,15) -0.7071 - 0.7071i<br>(15,15) 0.7071 + 0.7071i<br>(8,16) -0.9239 - 0.3827i<br>(16,16) 0.9239 + 0.3827i | $\approx$ | 1.0e+02 *<br>(1,1) -5.1200 + 0.0000i<br>(9,1) -5.1200 + 0.0000i<br>(2,2) -0.0000 + 0.0000i<br>(10,2) -0.0000 + 0.0000i<br>(3,3) -0.0100 + 0.0000i<br>(11,3) -0.0100 + 0.0000i<br>(4,4) -0.0001 + 0.0000i<br>(12,4) -0.0001 + 0.0000i<br>(5,5) -0.0025 + 0.0000i<br>(13,5) -0.0025 + 0.0000i<br>(6,6) -0.0002 + 0.0000i<br>(14,6) -0.0002 + 0.0000i<br>(7,7) -0.0003 + 0.0000i<br>(15,7) -0.0003 + 0.0000i<br>(8,8) -0.0000 + 0.0000i<br>(16,8) -0.0000 + 0.0000i<br>(1,9) 0.0003 + 0.0000i<br>(9,9) -0.0003 + 0.0000i<br>(2,10) 0.0001 + 0.0000i<br>(10,10) -0.0001 + 0.0000i<br>(3,11) -0.0000 + 0.0000i<br>(11,11) 0.0000 - 0.0000i<br>(4,12) 0.0002 + 0.0000i<br>(12,12) -0.0002 + 0.0000i<br>(5,13) 0.0000 + 0.0000i<br>(13,13) -0.0000 + 0.0000i<br>(6,14) 0.0002 + 0.0000i<br>(14,14) -0.0002 - 0.0000i<br>(7,15) 0.0000 + 0.0000i<br>(15,15) -0.0000 - 0.0000i<br>(8,16) -0.0001 + 0.0000i<br>(16,16) 0.0001 + 0.0000i | $\times$ | 1.0e+03 *<br>(1,1) 0.0010 + 0.0000i<br>(9,1) 0.0000 + 0.0000i<br>(2,2) 0.0020 + 0.0000i<br>(10,2) 0.1200 + 0.0000i<br>(3,3) 0.0005 + 0.0000i<br>(11,3) 0.0005 + 0.0000i<br>(4,4) 0.2560 + 0.0000i<br>(12,4) 0.0003 + 0.0000i<br>(5,5) 0.0010 + 0.0000i<br>(13,5) 0.0010 + 0.0000i<br>(6,6) 0.0320 + 0.0000i<br>(14,6) 0.0010 + 0.0000i<br>(7,7) 0.0003 + 0.0000i<br>(15,7) 0.0000 + 0.0000i<br>(8,8) -0.0000 - 0.0040i<br>(16,8) -0.0000 - 0.0000i<br>(10,10) 0.0010 + 0.0040i<br>(11,11) 0.0005 + 0.0010i<br>(12,12) -0.0000 + 0.0000i<br>(13,13) -0.0000 - 0.0040i<br>(14,14) -0.0000 - 0.0000i<br>(15,15) -0.0000 - 0.0000i<br>(16,16) -0.0003 - 0.0001i | $\times$ | 1.0e+02 *<br>(1,1) 0.0000 + 0.0000i<br>(9,1) 0.0003 + 0.0000i<br>(2,2) 0.0000 + 0.0000i<br>(10,2) 0.0000 + 0.0000i<br>(3,3) 0.0050 + 0.0000i<br>(11,3) 0.0002 + 0.0000i<br>(4,4) 0.0000 + 0.0000i<br>(12,4) 0.0400 + 0.0000i<br>(5,5) 0.0200 + 0.0000i<br>(13,5) 0.0200 + 0.0000i<br>(6,6) 0.0000 + 0.0000i<br>(14,6) 0.0400 + 0.0000i<br>(7,7) 0.0400 + 0.0000i<br>(15,7) -0.0100 + 0.0000i<br>(8,8) -0.0200 + 0.0000i<br>(9,9) -0.0100 + 0.0000i<br>(10,9) 0.0000 + 0.0000i<br>(11,9) 0.0000 + 0.0000i<br>(12,10) -0.0000 + 0.0000i<br>(13,10) -0.0000 - 0.1600i<br>(14,10) -0.0000 + 0.0000i<br>(15,10) -0.0000 + 0.0000i<br>(16,10) -0.0000 + 0.0000i<br>(12,12) -0.0400 + 0.0000i<br>(13,13) -0.0100 - 0.0000i<br>(14,14) -0.0100 - 0.0000i<br>(15,15) -2.5600 + 0.0000i<br>(16,15) -2.5600 + 5.1200i<br>(16,16) -0.0100 - 0.0000i | $\times$ | 1.0e+02 *<br>(1,1) -0.0050 + 0.0000i<br>(2,2) -0.0050 + 0.0000i<br>(3,3) -1.2800 + 0.0000i<br>(4,4) -0.0200 + 0.0000i<br>(5,5) -5.1200 + 0.0000i<br>(6,6) -1.2800 + 0.0000i<br>(7,7) -0.3200 + 0.0000i<br>(8,8) -0.0001 + 0.0000i<br>(9,9) -0.0100 + 0.0000i<br>(10,9) 0.0000 + 0.0000i<br>(11,9) 0.0000 + 0.0000i<br>(12,10) -0.0000 + 0.0000i<br>(13,10) -0.0000 + 0.0400i<br>(14,10) -0.0000 + 0.0000i<br>(15,11) -0.1600 + 0.0000i<br>(16,11) -0.0000 - 0.0000i<br>(12,12) 0.0000 + 0.0000i<br>(13,13) -0.0400 - 0.0200i<br>(14,14) -0.0100 + 0.0000i<br>(15,15) 0.0000 + 0.0000i<br>(16,15) 0.0000 + 0.0000i<br>(16,16) 5.1200 - 0.0050i |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| $A_3$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |           | $A_3^4$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |          | $A_3^3$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |          | $A_3^2$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |          | $A_3^1$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| (1,1) 1.0000 + 0.0000i<br>(5,1) 1.0000 + 0.0000i<br>(2,2) 1.0000 + 0.0000i<br>(6,2) 1.0000 + 0.0000i<br>(3,3) 1.0000 + 0.0000i<br>(7,3) 1.0000 + 0.0000i<br>(4,4) 1.0000 + 0.0000i<br>(8,4) 1.0000 + 0.0000i<br>(1,5) 1.0000 + 0.0000i<br>(5,5) -1.0000 + 0.0000i<br>(2,6) 0.7071 - 0.7071i<br>(6,6) -0.7071 + 0.7071i<br>(3,7) -0.0000 - 1.0000i<br>(7,7) 0.0000 + 1.0000i<br>(4,8) -0.7071 - 0.7071i<br>(8,8) 0.7071 + 0.7071i<br>(9,9) 1.0000 + 0.0000i<br>(13,9) 1.0000 + 0.0000i<br>(10,10) 1.0000 + 0.0000i<br>(14,10) 1.0000 + 0.0000i<br>(11,11) 1.0000 - 1.0000i<br>(15,11) 1.0000 + 0.0000i<br>(12,12) 1.0000 + 0.0000i<br>(16,12) 1.0000 + 0.0000i<br>(9,13) 1.0000 + 0.0000i<br>(13,13) -1.0000 + 0.0000i<br>(10,14) 0.7071 - 0.7071i<br>(14,14) -0.7071 + 0.7071i<br>(11,15) -0.0000 - 1.0000i<br>(15,15) 0.0000 + 1.0000i<br>(12,16) -0.7071 - 0.7071i<br>(16,16) 0.7071 + 0.7071i | $\approx$ | 1.0e+03 *<br>(1,1) 0.0010<br>(2,2) 0.0000<br>(3,3) 0.0000<br>(4,4) 0.0003<br>(5,5) -0.0000<br>(6,6) -0.1200<br>(7,7) -0.0000<br>(8,8) -0.0001<br>(9,9) 0.0000<br>(10,10) 0.0020<br>(11,11) 0.0000<br>(12,12) 0.0000<br>(13,13) -1.0240<br>(14,14) -0.0000<br>(15,15) -0.0000<br>(16,16) -0.0000                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\times$ | (1,1) -1.0000<br>(5,1) 64.0000<br>(2,2) -32.0000<br>(6,2) 0.0010<br>(3,3) -32.0000<br>(7,3) 32.0000<br>(4,4) -0.1250<br>(8,4) 0.5000<br>(1,5) 8.0000<br>(5,5) 512.0000<br>(2,6) 128.0000<br>(6,6) 0.0039<br>(3,7) -1.0000<br>(7,7) -1.0000<br>(4,8) -0.0020<br>(8,8) -0.0078<br>(9,9) -4.0000<br>(13,9) 0.0312<br>(10,10) -0.0078<br>(14,10) 8.0000<br>(11,11) -16.0000<br>(15,11) 16.0000<br>(12,12) -0.0625<br>(16,12) 0.1250<br>(9,13) 0.5000<br>(13,13) 0.0039<br>(10,14) -0.0312<br>(14,14) -32.0000<br>(11,15) -1.0000<br>(15,15) -1.0000<br>(12,16) -0.0156<br>(16,16) -0.0312                                                                       | $\times$ | 1.0e+03 *<br>(1,1) 0.2560 + 0.0000i<br>(2,2) 0.0020 + 0.0000i<br>(3,3) 0.1200 + 0.0000i<br>(4,4) 0.0001 + 0.0000i<br>(5,5) 0.0010 + 0.0000i<br>(6,6) -0.0010 + 0.0010i<br>(7,7) -0.0000 - 1.0240i<br>(8,8) -0.0320 - 0.0320i<br>(9,9) 0.0320 + 0.0000i<br>(10,10) 0.0160 + 0.0000i<br>(11,11) 0.2560 + 0.0000i<br>(12,12) 0.0160 + 0.0000i<br>(13,13) 0.0160 + 0.0000i<br>(14,14) -0.0000 + 0.0000i<br>(15,15) -0.0000 - 1.0240i<br>(16,16) -0.2560 - 0.2560i                                                                                                                                                                                                                                                                                                                                                                           | $\times$ | (1,1) -0.0039<br>(2,2) -4.0000<br>(3,3) -0.2500<br>(4,4) -512.0000<br>(5,5) 0.1250<br>(6,6) -1.0000<br>(7,7) -1.0000<br>(8,8) -32.0000<br>(9,9) -0.0010<br>(10,10) -4.0000<br>(11,11) -0.2500<br>(12,12) -32.0000<br>(13,13) 0.0156<br>(14,14) 1.0000<br>(15,15) -1.0000<br>(16,16) -4.0000                                                                                                                                                                                                                                                                                                                                                   |

图 8.10  $F_{16}$  的 C-T 基-2 分解矩阵  $A_4$ ,  $A_3$  的量化逼近

在精度下界。算法虽然避免了进行大规模整数规划探索，但也可能陷入局部最优。

## 附录 A 问题二附录

$$\mathbf{F}_{16} = \beta \mathbf{A}_4 \mathbf{A}_3 \mathbf{A}_2 \mathbf{A}_1$$







[illegible]

[illegible]



[illegible]

[illegible]



## 附录 C 主要实验代码

### 3.1 问题 2 代码

Faust.py

```
#!/usr/bin/env python
# coding: utf-8
import numpy as np
from numpy import linalg as la
import copy
from tools.butterfly import butterfly_S
from tools.constraint import prox_splincol
from tools.projection import projector
from tools.complex import complex_multiplications
from tools.tolatex import array_to_latex

#####
#                               AUXILIARY FUNCTIONS                               #
#####
def dft_matrix(N):
    """生成DFT矩阵"""
    n = np.arange(N)
    k = n.reshape((N, 1))
    W = np.exp(-2j * np.pi * k * n / N)
    return W

def non_square_id(n, m):
    M = np.matrix(np.zeros((n, m)))
    for i in range(min(n, m)):
        M[i, i] = 1
    return M

def left_product_matrices(S, order_id = 1):
    if len(S) == 0:
        return np.matrix(np.identity(order_id, dtype=np.complex128))

    res = S[0]
    for i in range(1, len(S)):
        res = S[i].dot(res)
```

```

    return res

#####
#                                     MAIN FUNCTIONS                                #
#####

def palm4MSA(A, J, constrs, Ss_init, lamb_init, N, projector_method
            = None, mu = 0.001):

    alpha = 1e-3
    S = copy.copy(Ss_init)
    lamb = copy.copy(lamb_init)
    for i in range(N):
        mu_step = mu + i/(N-1)*(0.1*mu - mu) # Linear decreasing of
            the learning rate from mu to 10% of mu
        for j in range(J):
            L = left_product_matrices(S[j+1:], A.shape[0]) # Parte
                izquierda a S[j]
            R = left_product_matrices(S[:j], A.shape[1])    # Parte
                derecha a S[j]

            L_norm = np.linalg.norm(L, ord=2)
            R_norm = np.linalg.norm(R, ord=2)

            L_norm = 1 if L_norm == 0 else L_norm
            R_norm = 1 if R_norm == 0 else R_norm

            min_c_value = (lamb * L_norm * R_norm) ** 2 # lipsitchz
                constant
            B = S[j] - 1./c * lamb * L.T @ (lamb * L @ S[j] @ R - A)
                @ R.T
            # B = S[j] - 1./c * lamb * L.T @ (lamb *
                left_product_matrices(S) - A) @ R.T

            # No normalization: S[j] = B
            S[j] = prox_splincol(B, constrs[j])
            # integer projector

```

```

        S[j] = projector(S[j],method=projector_method)
        # Rmk: If there are no requirements of sparsity, the
            code will be
        if projector_method is None:
            S[j] /= la.norm(S[j], 'fro')

    hat_A = left_product_matrices(S) # Resulting matrix

    numer = np.trace(A.T @ hat_A)
    denom = max(np.trace(hat_A.T @ hat_A), 10**(-10)) # Avoids
        division by zero
    lamb = lamb if numer/denom == 0 else numer/denom

    # lamb = Beta[np.argmin(error)]

    # lamb = update_scaling_factor(A, hat_A)
    # No normalization: lamb = 1
    # lamb = 1

    return S, lamb

def update_scaling_factor(X, X_est):
    return np.sum(X * X_est) / np.sum(X_est ** 2)

def create_default_Ss_raw(dims):
    n_rows, factors, n_cols = dims
    Ss = [np.matrix(np.random.rand(factors, n_cols)), np.matrix(np.
        random.rand(n_rows, factors))]
    return Ss

def test_result(S, lamb, R_test):
    Rp = lamb*left_product_matrices(S)
    frobenius_norm = np.sqrt(np.linalg.norm(R_test - Rp, 'fro')**2)
        /8
    return frobenius_norm

def create_default_Tree(dim, factors=3):
    root = np.linspace(factors,1,factors)
    tree = []

```

```

for i in range(0, factors-1):
    tree.append([butterfly_S(root[i], root[i], dim), butterfly_S(
        root[-1], root[i+1], dim)])
    # root = root[1:factors]
return tree

def FAuST(A, factors, constrs, constrs_tilde, N_local, N_global,
mus_local=None, projector_method = None, mus_global=None):

    T = A
    J = factors

    mus_local = [0.001 for _ in range(J-1)] if mus_local is None
    else mus_local
    mus_global = [0.001 for _ in range(J-1)] if mus_global is None
    else mus_global

    default_lamb = 1
    global_lamb = default_lamb
    Ss = []
    tree = create_default_Tree(T.shape[0], J)
    # Respecto a la notacion del paper: s = 1-1
    for s in range(J-1):
        print('*****')
        print('Deep: ' + str(s))

        ## LOCAL FACTORIZATION
        F, lamb = palm4MSA(T, 2, constrs[s], tree[s], default_lamb,
            N_local, projector_method, mus_local[s])
        F1, F2 = F

        frobenius_norm = test_result(F, lamb, T)
        print('Frobenius_norm after local factorization: ' + str(
            frobenius_norm))

        ## GLOBAL FACTORIZATION
        Ss.append(F1)
        T = lamb * F2
        Ss_init_global = Ss + [T]

```

```

        global_Ss, global_lamb = palm4MSA(A, s+2, constrs_tilde[s],
            Ss_init_global, lamb*global_lamb, N_global,
            projector_method, mus_global[s])
    T = global_Ss[-1]
    Ss = global_Ss[:-1]

    frobenius_norm = test_result(global_Ss, global_lamb, A)
    print('Frobenius_norm after global factorization: ' + str(
        frobenius_norm))

    Ss.append(T)

    return Ss, global_lamb

def create_default_Ss(dim):
    lst_S_init = []
    for i in reversed(range(1, int(np.log2(dim))+1)):
        lst_S_init.append(butterfly_S(i, i, dim))
    return lst_S_init

if __name__ == '__main__':

    F4 = dft_matrix(4)
    F8 = dft_matrix(8)
    F16 = dft_matrix(16)
    F32 = dft_matrix(32)
    from scipy.io import loadmat

    # 读取MAT文件
    mat_contents_F8 = loadmat('data/P8.mat')
    F8_A3 = mat_contents_F8['A3']
    F8_A2 = mat_contents_F8['A2']
    F8_A1 = mat_contents_F8['A1']
    F8_Pn = mat_contents_F8['Pn']

    mat_contents_F16 = loadmat('data/P16.mat')
    F16_A4 = mat_contents_F16['A4']
    F16_A3 = mat_contents_F16['A3']
    F16_A2 = mat_contents_F16['A2']

```



```

F16_A1 = mat_contents_F16['A1']
F16_Pn = mat_contents_F16['Pn']

mat_contents_F32 = loadmat('data/P32.mat')
F32_A5 = mat_contents_F32['A5']
F32_A4 = mat_contents_F32['A4']
F32_A3 = mat_contents_F32['A3']
F32_A2 = mat_contents_F32['A2']
F32_A1 = mat_contents_F32['A1']
F32_Pn = mat_contents_F32['Pn']

# #####F8#####
F8_true = F8_A3 @ F8_A2 @ F8_A1 @ F8_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F8-F8_true, 'fro')**2)/8
print('F8_true fro_norm=',frobenius_norm)

F8_A3_proj = projector(F8_A3,method='nearest')
F8_est = F8_A3_proj @ F8_A2 @ F8_A1 @ F8_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F8-F8_est, 'fro')**2)/8
print('F8_est fro_norm=',frobenius_norm)

# #####F16#####
F16_true = F16_A4 @ F16_A3 @ F16_A2 @ F16_A1 @ F16_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F16-F16_true, 'fro')**2)
/16
print('F16_true fro_norm=',frobenius_norm)

F16_A4_proj = projector(F16_A4,method='nearest')
F16_A3_proj = projector(F16_A3,method='nearest')
F16_est = F16_A4_proj @ F16_A3_proj @ F16_A2 @ F16_A1 @ F16_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F16-F16_est, 'fro')**2)
/16
print('F16_est fro_norm=',frobenius_norm)

# #####F32#####
F32_true = F32_A5 @ F32_A4 @ F32_A3 @ F32_A2 @ F32_A1 @ F32_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F32-F32_true, 'fro')**2)
/32
print('F32_true fro_norm=',frobenius_norm)

```

```

F32_A5_proj = projector(F32_A5,method='nearest')
F32_A4_proj = projector(F32_A4,method='nearest')
F32_A3_proj = projector(F32_A3,method='nearest')
F32_est = F32_A5_proj @ F32_A4_proj @ F32_A3_proj @ F32_A2 @
          F32_A1 @ F32_Pn
frobenius_norm = np.sqrt(np.linalg.norm(F32-F32_est, 'fro')**2)
              /32
print('F32_est fro_norm=',frobenius_norm)

##### TEST PALM4MSA
#####
d = 8
factor = int(np.log2(d))
# factor = 1
lst_S_init = create_default_Ss(d)

# lst_S_init = []
# for i in range(factor):
#     lst_S_init.append(np.eye((d),dtype=np.complex128))

S, lamb = palm4MSA(A = F8, J = factor, constrs=8*np.ones(factor)
, Ss_init=lst_S_init, lamb_init=1, N=1000, projector_method =
None, mu=0.001)
# for i in range(len(S)):
#     S[i] = projector(S[i],method='nearest')

Approximate = left_product_matrices(S)
# complexity = complex_multiplications(S, lamb)
# print('complexity = ',complexity)
# frobenius_norm = np.sqrt(np.linalg.norm(F4 - lamb *
Approximate, 'fro')**2)/d
# print('frobenius_norm = ',frobenius_norm)
# print('lambda = ',lamb)

# F8_est = lamb * Approximate @ F8_A2 @ F8_A1 @ F8_Pn
# frobenius_norm = np.sqrt(np.linalg.norm(F8 - F8_est, 'fro')
**2)/d
# print('frobenius_norm = ',frobenius_norm)

```

```

# print('lambda = ', lamb)

##### TEST FAUST #####
# constrs = [[8,8],[8,8]]
# constrs_tilde = [[8,8],[8,8,8]]
# Ss, global_lamd = FAuST(F8, 3, constrs, constrs_tilde, 500,
    500, projector_method='nearest')
# Approximate = left_product_matrices(Ss)
# frobenius_norm = np.sqrt(np.linalg.norm(F8 - global_lamd *
    Approximate, 'fro')**2)/8
# print('frobenius_norm=', frobenius_norm)
# print('lambda=', global_lamd)

```

### 3.2 问题3 代码

#### Faust.py

```

import numpy as np
from scipy.io import loadmat
from tools.matlab_supp import dft_matrix, hierarchical,
    create_default_Tree, multiply_list, grad_opt_beta
from tools.complex import complex_multiplications
from tools.tolatex import array_to_latex

##### load data #####
F4 = dft_matrix(4)
F8 = dft_matrix(8)
F16 = dft_matrix(16)
F32 = dft_matrix(32)
from scipy.io import loadmat

mat_contents_F4 = loadmat('data/F4.mat')
F4_A2 = mat_contents_F4['A2']
F4_A1 = mat_contents_F4['A1']
F4_Pn = mat_contents_F4['Pn']

# 读取MAT文件
mat_contents_F8 = loadmat('data/P8.mat')
F8_A3 = mat_contents_F8['A3']
F8_A2 = mat_contents_F8['A2']

```

```

F8_A1 = mat_contents_F8['A1']
F8_Pn = mat_contents_F8['Pn']

mat_contents_F16 = loadmat('data/P16.mat')
F16_A4 = mat_contents_F16['A4']
F16_A3 = mat_contents_F16['A3']
F16_A2 = mat_contents_F16['A2']
F16_A1 = mat_contents_F16['A1']
F16_Pn = mat_contents_F16['Pn']

mat_contents_F32 = loadmat('data/P32.mat')
F32_A5 = mat_contents_F32['A5']
F32_A4 = mat_contents_F32['A4']
F32_A3 = mat_contents_F32['A3']
F32_A2 = mat_contents_F32['A2']
F32_A1 = mat_contents_F32['A1']
F32_Pn = mat_contents_F32['Pn']

# N = 4 # N是DFT矩阵维度
# J = np.log2(N) # J是DFT矩阵改被分为多少个子矩阵

# tree_F4 = create_default_Tree(N, factors=J)

# X_0 = []
# T = F4@np.linalg.inv(F4_Pn.T)
# for i in range(len(tree_F4)):
#     ML, MR = hierarchical(T, tree_F4[i][0], tree_F4[i][1], flag=2,
#         project_method='nearest')
#     X_0.append(ML)
#     T = MR
#     if i == len(tree_F4)-1:
#         X_0.append(MR)
#     break

# print("##### N=4 #####")
# # Beta_1 = grad_opt_beta(F8, X_1[0] @ X_1[1] @ X_1[2] @ F8_A2 @
#     F8_A1 @ F8_Pn)
# # F8_est = Beta_1 * X_1[0] @ X_1[1] @ X_1[2] @ F8_A2 @ F8_A1 @
#     F8_Pn

```

```

# Beta_0 = grad_opt_beta(F4@np.linalg.inv(F4_Pn.T), X_0[0] @ X_0[1])
# F8_est = Beta_0 * X_0[0] @ X_0[1]
# error = np.linalg.norm(F4@np.linalg.inv(F4_Pn.T) - F8_est, 'fro')
# / N
# print("F8 fro_norm = ",error)
# print("Beta = ",Beta_0)
# complexity = complex_multiplications([X_0[1],X_0[0]], Beta_0)
# print("complexity = ",complexity)

##### N=8 parameters #####
N = 8 # N是DFT矩阵维度
J = np.log2(N) # J是DFT矩阵改被分为多少个子矩阵

tree_F8 = create_default_Tree(N,factors=J)

X_1 = []
T = F8@np.linalg.inv(F8_Pn)
for i in range(len(tree_F8)):
    ML, MR = hierarchical(T, tree_F8[i][0],tree_F8[i][1], flag=2,
        project_method='nearest')
    X_1.append(ML)
    T = MR
    if i == len(tree_F8)-1:
        X_1.append(MR)
        break

print("##### N=8 #####")
Beta_1 = grad_opt_beta(F8@np.linalg.inv(F8_Pn), X_1[0] @ X_1[1] @
    X_1[2])
F8_est = Beta_1 * X_1[0] @ X_1[1] @ X_1[2] @ F8_Pn
# Beta_1 = grad_opt_beta(F8@np.linalg.inv(F8_Pn.T), X_1[0] @ X_1[1]
    @ X_1[2])
# F8_est = Beta_1 * X_1[0] @ X_1[1] @ X_1[2]
error = np.linalg.norm(F8 - F8_est, 'fro') / N
print("F8 fro_norm = ",error)
print("Beta = ",Beta_1)
complexity = complex_multiplications([ F8_Pn,X_1[2],X_1[1],X_1[0]],
    Beta_1)
print("complexity = ",complexity)

```

```

##### N=16 parameters #####
N = 16 # N是DFT矩阵维度
J = np.log2(N) # J是DFT矩阵改被分为多少个子矩阵

tree_F16 = create_default_Tree(N, factors=J)

X_2 = []
T = F16@np.linalg.inv(F16_Pn)
for i in range(len(tree_F16)):
    ML, MR = hierarchical(T, tree_F16[i][0], tree_F16[i][1], flag=2,
        project_method='nearest')
    X_2.append(ML)
    T = MR
    if i == len(tree_F16)-1:
        X_2.append(MR)
        break

X_3 = []
T = F16_A3
for i in range(len(tree_F16)):
    ML, MR = hierarchical(T, tree_F16[i][0], tree_F16[i][1], flag=2,
        project_method='nearest')
    X_3.append(ML)
    T = MR
    if i == len(tree_F16)-1:
        X_3.append(MR)
        break

print("##### N=16 #####")
# Beta_3 = grad_opt_beta(F16_A3, X_3[0] @ X_3[1] @ X_3[2] @ X_3[3])
Beta_2 = grad_opt_beta(F16@np.linalg.inv(F16_Pn), X_2[0] @ X_2[1] @
    X_2[2] @ X_2[3])
# F16_A3_est = Beta_3 * X_3[0] @ X_3[1] @ X_3[2] @ X_3[3]
F16_A4_est = Beta_2 * X_2[0] @ X_2[1] @ X_2[2] @ X_2[3]
F16_est = F16_A4_est @ F16_Pn
error = np.linalg.norm(F16 - F16_est, 'fro') / N

```

```

beta = multiply_list([Beta_2])
print("Beta = ",beta)
print("F8 fro_norm = ",error)
complexity = complex_multiplications([X_2[0] @ X_2[1] @ X_2[2] @ X_2
    [3] @ F16_Pn], Beta_2)
print("complexity = ",complexity)

##### N=32 parameters #####
N = 32 # N是DFT矩阵维度
J = np.log2(N) # J是DFT矩阵改被分为多少个子矩阵

tree_F32 = create_default_Tree(N,factors=J)

X_4 = []
T = F32@np.linalg.inv(F32_Pn)
for i in range(len(tree_F32)):
    ML, MR = hierarchical(T, tree_F32[i][0],tree_F32[i][1], flag=2,
        project_method='nearest')
    X_4.append(ML)
    T = MR
    if i == len(tree_F32)-1:
        X_4.append(MR)
        break

X_5 = []
T = F32_A4
for i in range(len(tree_F32)):
    ML, MR = hierarchical(T, tree_F32[i][0],tree_F32[i][1], flag=2,
        project_method='nearest')
    X_5.append(ML)
    T = MR
    if i == len(tree_F32)-1:
        X_5.append(MR)
        break

X_6 = []
T = F32_A3
for i in range(len(tree_F32)):
    ML, MR = hierarchical(T, tree_F32[i][0],tree_F32[i][1], flag=2,

```

```

        project_method='nearest')
X_6.append(ML)
T = MR
if i == len(tree_F32)-1:
    X_6.append(MR)
    break

print("##### N=32 #####")
Beta_4 = grad_opt_beta(F32@np.linalg.inv(F32_Pn), X_4[0] @ X_4[1] @
    X_4[2] @ X_4[3] @ X_4[4])
# Beta_5 = grad_opt_beta(F32_A4, X_5[0] @ X_5[1] @ X_5[2] @ X_5[3]
    @ X_5[4])
# Beta_6 = grad_opt_beta(F32_A3, X_6[0] @ X_6[1] @ X_6[2] @ X_6[3]
    @ X_6[4])

# F32_A3_est = Beta_6 * X_6[0] @ X_6[1] @ X_6[2] @ X_6[3] @ X_6[4]
# error1 = np.linalg.norm(F32_A3 - F32_A3_est, 'fro') / N
# print("F32_A3 fro_norm = ",error1)

# F32_A4_est = Beta_5 * X_5[0] @ X_5[1] @ X_5[2] @ X_5[3] @ X_5[4]
# error2 = np.linalg.norm(F32_A4 - F32_A4_est, 'fro') / N
# print("F32_A4 fro_norm = ",error2)

F32_est = Beta_4 * X_4[0] @ X_4[1] @ X_4[2] @ X_4[3] @ X_4[4] @
    F32_Pn
# error3 = np.linalg.norm(F32_A5 - F32_A5_est, 'fro') / N
# print("F32_A5 fro_norm = ",error3)

# F32_est = F32_A5_est @ F32_A4_est @ F32_A3_est @ F32_A2 @ F32_A1 @
    F32_Pn

error = np.linalg.norm(F32 - F32_est, 'fro') / N

beta = multiply_list([Beta_4])
print("Beta = ",beta)
print("F8 fro_norm = ",error)
complexity = complex_multiplications([X_4[0] @ X_4[1] @ X_4[2] @ X_4
    [3] @ X_4[4] @ F32_Pn], beta)
print("complexity = ",complexity)

```



### 3.3 问题4代码

#### 遍历性定理逼近求解.m

```
%遍历性定理逼近求解
clear
clc
angle_init = [0 0;pi/2 0;pi 0;-pi/2 0;atan(1/2) atan(-1/2);atan(2) atan(1/2);atan(1/2)+pi/2 atan(-1/2);atan(1/4) atan(-1/4); atan(4) atan(1/4);atan(1/4)+pi/2 atan(-1/4)]; %initial angle
angle_rotate = [atan(1/4) -atan(1/4);atan(4) atan(1/4);atan(1/2) -atan(1/2);atan(2) atan(1/2)];%rotation angle
angle_diff = [pi/4,pi/4*3,pi/4+pi,2*pi-pi/4];%angle difference
angle_target_mod90 = [0,atan(1/4),atan(1/2),pi/4,atan(2),pi/2];%target angle
global_min_error = 0.076773;
for n = 1:1000%number of rotation matrices
    for in_init = 1:10 %1:10
        angles = angle_init(in_init,:);
        for in_rotate = 4 %1:4
            delta_angles = angle_rotate(in_rotate,:);
            %disp(['New parameters: in_init =' num2str(in_init) '
                in_rotate=' int2str(in_rotate)])

            temp_ang1 = angles(1)+n*delta_angles(1);
            temp_ang2 = angles(2)+n*delta_angles(2);
            temp_diff = mod(temp_ang2-temp_ang1,2*pi);
            [diff_error,diff_index ]= min(abs(temp_diff-angle_diff));
            [target_error,target_index ]= min(abs(mod(temp_ang1,pi/2)-angle_target_mod90));
            total_error = diff_error+target_error;
            if total_error < global_min_error
                global_min_error = total_error;
                disp('NEW GLOBAL BEST
                    .....
                    ')
                disp(['initial angle: ' num2str(in_init)])
                disp(['rotation angle: ' num2str(in_rotate)])
                disp(['n: ' num2str(n)])
                disp(['total error: ' num2str(total_error)])
```

```

disp(['diff_error: ' num2str(diff_error)])
disp(['diff_index: ' num2str(diff_index)])
disp(['target_error: ' num2str(target_error)])
disp(['target_index: ' num2str(target_index)])

end

end

end

end

%1e4:0.07096

```

### 3.4 问题5 代码

main.m

```

clc,clear,close all
J = 3;
N = 2^J;
A = dftmtx(N);
ctfft(J);
p = 1;
q = J;
l = 1;
% [ML,MR,beta] = Hierarchical_ga(A*Pn,p,q,l,N,0);
[ML,MR,beta] = Hierarchical_ga(A3,p,q,l,N,1);
X{1} = ML;
Beta(1) = beta;
p = l+1;
q = J;
l = p;
[ML,MR,beta] = Hierarchical_ga(MR,p,q,l,N,0);
X{2} = ML;
X{3} = MR;
Beta(2) = beta;
% X{1} = lh(X{1});
% X{2} = lh(X{2});
% X{3} = lh(X{3});
beta = gradOptBeta(A,X{1}*X{2}*X{3}*A2*A1*Pn');
% beta = gradOptBeta(A,X{1}*X{2}*X{3}*Pn)
norm(A - beta*Beta(1)*Beta(2)*X{1}*X{2}*X{3}*A2*A1*Pn',"fro")/N
% norm(A - beta*Beta(1)*Beta(2)*X{1}*X{2}*X{3}*Pn',"fro")/N

```

```

function X = lh(X)
[m,n] = size(X);
for i = 1:n
X(:,i) = ProjRealImag(X(:,i));
end
end

```

### Hierarchical\_ga.m

```

function [ML,MR,beta] = Hierarchical_ga(M,p,q,l,N,flag)
SbfL = Sbf(p,l,N);
SbfR = Sbf(l+1,q,N);
[ML,MR,beta] = FSMF_ga(M,SbfL,SbfR',flag);
MR = MR';
end

```

### FSMF\_ga.m

```

function [X,Y,beta] = FSMF_ga(Z,SL,SR,flag)
beta = 1;
[m,r] = size(SL);
X = zeros(m,r);
Y = zeros(m,r);
S = phi(SL, SR); % S is a cell
for i = 1:r
% 对应左, 右模板分别为SL(:,i),SR(:,i)' 直接就是列向量
[indLrow,indLcol,valL] = find(SL(:,i)); %总数
[indRrow,indRcol,valR] = find(SR(:,i)); %
NL = length(indLrow);
NR = length(indRrow);
[u,sigma,v] = svd(Z.*S{i});
srealu = sparse(sign(real(u(:,1)))));
simagu = sparse(sign(imag(u(:,1)))));
srealv = sparse(sign(real(sigma(1,1)*v(:,1)))));
simagv = sparse(sign(imag(sigma(1,1)*v(:,1)))));
N = NL + NR; % 待优化个数
% 目标函数使得frobenius最小
f = @(x) norm(Z.*S{i} - (srealu.*sparse(indLrow,indLcol,2.^x(1:NL),m,1) +
    simagu.*sparse(indLrow,indLcol,2.^x(NL+1:2*NL)*1i,m,1))*...
(srealv.*sparse(indRrow,indRcol,2.^x(2*NL+1:2*NL+NR),m,1)+simagv.*
    sparse(indRrow,indRcol,2.^x(2*NL+1+NR:2*N)*1i,m,1))', 'fro');

```

```

% 建立整数遗传算法, 2^t次方逼近
intN = [1:1:2*N]';
[x, fval, ef] = ga(f, 2 * N, [], [], [], [], -10*ones(2*N,1), 10*ones(2*N,1), [], intN)
% [x, fval, ef] = ga(f, 2 * N, [], [], [], [], [], [], [], [], intN)
% Z = Z - signu.*sparse(indLrow,indLcol,2.^x(1:Nl)+2.^x(Nl+1:2*Nl)*1i,m,1)*...
% signv.*sparse(indRrow,indRcol,2.^x(2*Nl+1:2*Nl+Nr)+2.^x(2*Nl+1+Nr:2*N)*1i,m,1)';
X(:,i) = full((srealu.*sparse(indLrow,indLcol,2.^x(1:Nl),m,1)+simagu.*sparse(indLrow,indLcol,2.^x(Nl+1:2*Nl)*1i,m,1)));
Y(:,i) = full((srealv.*sparse(indRrow,indRcol,2.^x(2*Nl+1:2*Nl+Nr),m,1)+simagv.*sparse(indRrow,indRcol,2.^x(2*Nl+1+Nr:2*N)*1i,m,1)));
Z = Z - X(:,i)*Y(:,i)';
end
end
function K = phi(X,Y)
[~,r] = size(X);
K = cell(r,1);
for i = 1:r
K{i} = X(:,i)*Y(:,i)';
end
end
function [x,y] = minFZ(A,x,y)
lambda = 0.01;
ind = 1;
while (ind < 100)
dx = (x*y'-A)*y;
dy = (x*y'-A)'*x;
x = x - lambda * dx;
y = y - lambda * dy;
error = norm(A-x*y','fro');
if error < 0.1
break
end
ind = ind + 1;
end
end

```

### gradOptBeta.m

```
function beta = gradOptBeta(Z,Zhat)
    Beta = 0:0.01:5;
    [N, ~] = size(Z);
    error = zeros(length(Beta),1);
    for i = 1:length(Beta)
        beta = Beta(i);
        error(i) = norm(Z - beta*Zhat,'fro')/N;
    end
    figure
        plot(error)
    [~,ind] = min(error);
    beta = Beta(ind);
end
```

### 遍历性定理逼近求解.m

```
%第五问求解
clear
clc
angle_init = 0; %initial angle
angle_rotate = [atan(1/4) atan(1/2) atan(4) atan(2)];%rotation angle
angle_target_mod90 = [pi/2-pi/8,pi/8];%target angle
global_min_error = 1;
for n = 1:1000%number of rotation matrices
    angles = angle_init;
    for in_rotate = 1:4
        delta_angles = angle_rotate(in_rotate);
        %disp(['New parameters: in_init =' num2str(in_init) '
            in_rotate=' int2str(in_rotate)])

        temp_ang1 = angles+n*delta_angles;
        [target_error,target_index] = min(abs(mod(temp_ang1,pi/2)-
            angle_target_mod90));
        if target_error < global_min_error
            global_min_error = target_error;
            disp('NEW GLOBAL BEST
                .....
            ')
            disp(['rotation angle: ' num2str(in_rotate)])
```

```
        disp(['n: ' num2str(n)])
        disp(['target_error: ' num2str(target_error)])
        disp(['target_index: ' num2str(target_index)])
    end
end
end
%1e4:0.07096
```

## 参考文献

- [1] Suarez D, Cintra R J, Bayer F M, et al., Multi-beam RF aperture using multiplierless FFT approximation, *Electronics Letters*, 50(24):1788-1790, 2014.
- [2] Chabiron O, Malgouyres F, Tourneret J Y, et al., Toward fast transform learning, *International Journal of Computer Vision*, 114(2-3):195-216, 2015.
- [3] Le Q T, Riccietti E, Gribonval R, Spurious Valleys, NP-Hardness, and Tractability of Sparse Matrix Factorization with Fixed Support, *SIAM J. Matrix Anal. Appl.*, 44(2):503-529, 2023.
- [4] Dao T, Gu A, Eichhorn M, et al., Learning Fast Algorithms for Linear Transforms Using Butterfly Factorizations, *Proceedings of the 36th International Conference on Machine Learning*, PMLR, 1517-1527, 2019.
- [5] Van Loan C, *Frontiers in Applied Mathematics: vol. 10 Computational Frameworks for the Fast Fourier Transform*, Philadelphia: SIAM, 1992.
- [6] Le Magoarou L, Gribonval R, Flexible Multilayer Sparse Approximations of Matrices and Applications, *IEEE J. Sel. Top. Signal Process.*, 10(4):688-700, 2016.
- [7] Bolte J, Sabach S, Teboulle M, Proximal alternating linearized minimization for nonconvex and nonsmooth problems, *Mathematical Programming*, 146(1-2):459-494, 2014.
- [8] Zheng L, Riccietti E, Gribonval R, Efficient Identification of Butterfly Sparse Matrix Factorizations, *SIAM Journal on Mathematics of Data Science*, 5(1):22-49, 2023.